

# Homework 8: Fine-Tuning Pretrained Models

Joao De Oliveira

## Overview

In this lab we work with the **Financial Phrasebank** dataset — short financial news snippets annotated by experts for sentiment (negative / neutral / positive). We address Option 1: studying how LSTM performance and optimal hyperparameters vary with network width, using BERT pre-loaded embeddings and Optuna for hyperparameter tuning. The extra credit section then analyzes which texts are intrinsically hard to classify.

## Setup

```
In [2]: !pip install -q torchmetrics torchinfo pytorch_lightning ISLP
!pip install -q transformers datasets trl peft bitsandbytes optuna
!pip install -q scikit-learn

Preparing metadata (setup.py) ... done
_____ 983.4/983.4 kB 26.1 MB/s eta 0:00:00
_____ 852.2/852.2 kB 60.8 MB/s eta 0:00:00
_____ 16.6/16.6 MB 70.8 MB/s eta 0:00:00
_____ 409.1/409.1 kB 36.8 MB/s eta 0:00:00
_____ 84.6/84.6 kB 9.7 MB/s eta 0:00:00
_____ 117.3/117.3 kB 12.3 MB/s eta 0:00:00
Building wheel for autograd-gamma (setup.py) ... done
_____ 751.0/751.0 kB 19.7 MB/s eta 0:00:00
_____ 529.0/529.0 kB 40.3 MB/s eta 0:00:00
_____ 60.7/60.7 MB 14.8 MB/s eta 0:00:00
_____ 419.5/419.5 kB 37.4 MB/s eta 0:00:00
_____ 48.9/48.9 MB 18.7 MB/s eta 0:00:00
```

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
import logging
import shutil
import gc

import torch
from torch import nn
from torch.optim import Adam
from torch.utils.data import TensorDataset, DataLoader

import pytorch_lightning as pl
from pytorch_lightning import Trainer, seed_everything
from pytorch_lightning.loggers import CSVLogger
from pytorch_lightning.callbacks import (
    TQDMProgressBar, ModelCheckpoint, EarlyStopping, Callback
)
from torchinfo import summary

from transformers import AutoTokenizer, AutoModel
from datasets import load_dataset
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import optuna
from optuna.visualization import (
    plot_param_importances, plot_contour, plot_slice, plot_parallel_coordinate
)
import plotly.colors as pc

from ISLP.torch import SimpleDataModule, SimpleModule, ErrorTracker, rec_num_workers

seed_everything(0, workers=True)
torch.use_deterministic_algorithms(True, warn_only=True)

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
```

```
INFO:lightning_fabric.utilities.seed:Seed set to 0
Using device: cuda
```

## Data Loading and Splitting

We load the Financial Phrasebank dataset from Hugging Face and split the test set 50/50 into a validation set (used during training to monitor progress and guide early stopping) and a held-out test set (used only for final accuracy evaluation). This approach gives us roughly a 1% standard error on test accuracy and maximizes the amount of training data, as recommended in the lab instructions.

```
In [5]: ds = load_dataset("FinanceMTEB/financial_phrasebank")

train_data = ds["train"]
test_data = ds["test"]

train_texts = train_data["text"]
train_labels = train_data["label"]
test_texts = test_data["text"]
test_labels = test_data["label"]

print(f"Training samples : {len(train_texts)}")
print(f"Test samples      : {len(test_texts)}")
print(f"Label distribution (train): {pd.Series(train_labels).value_counts().to_dict()}")

# split test set 50/50 into val and held out test
test_idx_val, test_idx_test = train_test_split(
    range(len(test_texts)),
    test_size=0.5,
    stratify=test_labels,
    random_state=9999
)

val_texts = [test_texts[i] for i in test_idx_val]
val_labels = [test_labels[i] for i in test_idx_val]
final_test_texts = [test_texts[i] for i in test_idx_test]
final_test_labels = [test_labels[i] for i in test_idx_test]

print(f"\nValidation set : {len(val_texts)} samples")
print(f"Final test set : {len(final_test_texts)} samples")
```

```
Training samples : 1264
Test samples      : 1000
Label distribution (train): {1: 779, 2: 320, 0: 165}
```

```
Validation set : 500 samples
Final test set : 500 samples
```

## Pre-Loading BERT Embeddings

As recommended in the vignette, we "pre-load" BERT embeddings by running the frozen embedding layer over all samples once and saving the resulting tensors. This prevents fine-tuning the embedding but eliminates a costly embedding step on every forward pass during training, dramatically speeding up each Optuna trial. The BERT embedding dimension is 768.

```
In [6]: tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

train_tokens = tokenizer(list(train_texts), padding=True, truncation=True,
                        max_length=128, return_tensors="pt")
val_tokens = tokenizer(list(val_texts), padding=True, truncation=True,
                      max_length=128, return_tensors="pt")
test_tokens = tokenizer(list(final_test_texts), padding=True, truncation=True,
                      max_length=128, return_tensors="pt")

# load BERT, push to GPU, compute embeddings and then free it
bert = AutoModel.from_pretrained("bert-base-uncased").to(device)
bert.eval()

with torch.no_grad():
    train_embeds = bert.embeddings(train_tokens["input_ids"].to(device)).cpu()
    val_embeds = bert.embeddings(val_tokens["input_ids"].to(device)).cpu()
    test_embeds = bert.embeddings(test_tokens["input_ids"].to(device)).cpu()
```

```

del bert
gc.collect()
torch.cuda.empty_cache()

print(f"Train embeddings shape: {train_embeds.shape}")
print(f"Val embeddings shape: {val_embeds.shape}")
print(f"Test embeddings shape: {test_embeds.shape}")
print("BERT embeddings pre-loaded and BERT model freed from GPU memory.")

config.json: 0%|          | 0.00/570 [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
model.safetensors: 0%|          | 0.00/440M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
-----+-----+
cls.seq_relationship.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.predictions.transform.dense.weight | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Train embeddings shape: torch.Size([1264, 128, 768])
Val embeddings shape: torch.Size([500, 77, 768])
Test embeddings shape: torch.Size([500, 80, 768])
BERT embeddings pre-loaded and BERT model freed from GPU memory.

```

```

In [7]: seq_train_emb = TensorDataset(train_embeds, torch.tensor(train_labels, dtype=torch.long))
seq_val_emb = TensorDataset(val_embeds, torch.tensor(val_labels, dtype=torch.long))
seq_test_emb = TensorDataset(test_embeds, torch.tensor(final_test_labels, dtype=torch.long))

seq_dm_emb = SimpleDataModule(
    seq_train_emb, seq_test_emb,
    validation=seq_val_emb,
    batch_size=64,
    num_workers=2
)

```

## Helper Classes and Utilities

```

In [8]: class PrintMetrics(Callback):
    """Prints a one-line summary per epoch for easy progress tracking."""
    def on_validation_epoch_end(self, trainer, pl_module):
        m = trainer.callback_metrics
        epoch = trainer.current_epoch
        if all(k in m for k in ['train_loss', 'valid_loss', 'train_accuracy', 'valid_accuracy']):
            print(f"Epoch {epoch:3d} | "
                  f"train_loss: {m['train_loss'].item():.4f} | "
                  f"train_acc: {m['train_accuracy'].item():.4f} | "
                  f"valid_loss: {m['valid_loss'].item():.4f} | "
                  f"valid_acc: {m['valid_accuracy'].item():.4f}")

    def summary_plot(results, ax, col='loss', valid_legend='Validation',
                     training_legend='Training', ylabel='Loss'):
        col_train = f'train_{col}' if col == 'loss' else f'train_{col}_epoch'
        col_valid = f'valid_{col}'
        for column, color, label in zip(
            [col_train, col_valid], ['black', 'red'], [training_legend, valid_legend]
        ):
            subset = results.dropna(subset=[column])
            subset.plot(x='epoch', y=column, label=label, marker='o', color=color, ax=ax)
        ax.set_xlabel('Epoch')
        ax.set_ylabel(ylabel)
        return ax

```

## LSTM Model Architecture

We define `LSTMPreEmbedded`, a flexible LSTM classifier that accepts pre-computed BERT embeddings (dimension 768) as input. The model supports variable `hidden_size`, `num_layers`, and `dropout`, making it ideal for the Optuna sweep. The LSTM is bidirectional, concatenating forward and backward hidden states for the final classification.

```
In [11]: class LSTMPreEmbedded(nn.Module):
    """
    Bidirectional LSTM classifier operating on pre-computed BERT embeddings.
    Parameters:
        hidden_size - number of units per LSTM direction
        num_layers   - number of stacked LSTM layers
        dropout      - dropout probability (applied after embedding and after LSTM)
    """
    def __init__(self, hidden_size: int, num_layers: int, dropout: float):
        super().__init__()
        self.drop1 = nn.Dropout(dropout)
        self.lstm = nn.LSTM(
            input_size=768,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=True,
            dropout=dropout if num_layers > 1 else 0.0
        )
        self.drop2 = nn.Dropout(dropout)
        output_size = hidden_size * 2
        self.dense = nn.Linear(output_size, 3)

    def forward(self, x):
        embedded = self.drop1(x)
        val, (h_n, c_n) = self.lstm(embedded)
        forward_out = h_n[-2]
        backward_out = h_n[-1]
        combined = torch.cat((forward_out, backward_out), dim=1)
        return self.dense(self.drop2(combined))

# quick check
_demo = LSTMPreEmbedded(hidden_size=64, num_layers=2, dropout=0.1)
summary(_demo, input_data=train_embs[:4], col_names=['input_size', 'output_size', 'num_params'])
```

```
Out[11]: =====
=====
Layer (type:depth-idx)          Input Shape          Output Shape          Param #
=====
LSTMPreEmbedded                [4, 128, 768]        [4, 3]                --
├─Dropout: 1-1                  [4, 128, 768]        [4, 128, 768]         --
├─LSTM: 1-2                     [4, 128, 768]        [4, 128, 128]         526,336
├─Dropout: 1-3                  [4, 128]             [4, 128]              --
├─Linear: 1-4                   [4, 128]             [4, 3]                387
=====
Total params: 526,723
Trainable params: 526,723
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 269.49
=====
Input size (MB): 497.03
Forward/backward pass size (MB): 0.52
Params size (MB): 2.11
Estimated Total Size (MB): 499.66
=====
=====
```

## Problem 1: Finding Diminishing Returns for LSTM

### Approach

We sweep over six network widths: **4, 8, 32, 256, and 1028** (the set suggested in the lab). For each width we fix a single LSTM depth (2 layers) and run an Optuna study with 15 trials, tuning:

- `learning_rate` — controls step size during gradient descent

- `weight_decay` — L2 regularization strength
- `dropout` — dropout probability at both dropout layers
- `patience` — early stopping patience (in lieu of a fixed epoch budget)

We use `valid_accuracy` as the Optuna objective and `valid_loss` for checkpointing. After all widths are done we collect and display the best parameters and test accuracy for each width.

```
In [8]: warnings.filterwarnings("ignore")
logging.getLogger("pytorch_lightning").setLevel(logging.ERROR)

# store results for analysis
width_results = []

WIDTHS      = [4, 8, 32, 256, 1028]
NUM_LAYERS  = 2
N_TRIALS    = 15
def make_objective(hidden_size):
    """Return an Optuna objective function for a fixed hidden_size."""
    def objective(trial):
        lr      = trial.suggest_float("learning_rate", 5e-5, 5e-1, log=True)
        wd      = trial.suggest_float("weight_decay", 1e-5, 1e-1, log=True)
        dropout = trial.suggest_float("dropout", 0.05, 0.40)
        patience = trial.suggest_int("patience", 5, 30)

        model    = LSTMPreEmbedded(hidden_size=hidden_size, num_layers=NUM_LAYERS, dropout=dropout)
        optimizer = Adam(model.parameters(), lr=lr, weight_decay=wd)
        module    = SimpleModule.classification(model, optimizer=optimizer, num_classes=3)

        ckpt_dir = f'ckpt/lstm_w{hidden_size}_t{trial.number}'
        shutil.rmtree(ckpt_dir, ignore_errors=True)

        checkpoint_cb = ModelCheckpoint(
            monitor='valid_loss', mode='min', save_top_k=1, dirpath=ckpt_dir)
        early_stop_cb = EarlyStopping(
            monitor='valid_accuracy', mode='max', patience=patience)

        trainer = Trainer(
            deterministic=True,
            max_epochs=200,
            logger=False,
            enable_progress_bar=False,
            enable_model_summary=False,
            callbacks=[ErrorTracker(), checkpoint_cb, early_stop_cb]
        )
        trainer.fit(module, datamodule=seq_dm_emb)
        test_out = trainer.test(module, datamodule=seq_dm_emb, ckpt_path='best', verbose=False)
        val_acc = test_out[0]['test_accuracy']

        print(f" Trial {trial.number}: acc={val_acc:.4f} | "
              f"lr={lr:.2e} wd={wd:.2e} dropout={dropout:.3f} patience={patience}")

        shutil.rmtree(ckpt_dir, ignore_errors=True)
        del model, module, trainer
        gc.collect()
        torch.cuda.empty_cache()
        return val_acc

    return objective

# Main sweep
all_studies = {}

for width in WIDTHS:
    print(f"\n{'='*60}")
    print(f" Optuna study: hidden_size = {width}")
    print(f"{'='*60}")

    study = optuna.create_study(direction="maximize",
                                sampler=optuna.samplers.TPESampler(seed=42))
    study.optimize(make_objective(width), n_trials=N_TRIALS, show_progress_bar=False)

    best = study.best_params
    best_acc = study.best_value
    all_studies[width] = study
```

```

width_results.append({
    "hidden_size": width,
    "best_val_acc": best_acc,
    "learning_rate": best["learning_rate"],
    "weight_decay": best["weight_decay"],
    "dropout": best["dropout"],
    "patience": best["patience"],
})

print(f" → Best val accuracy : {best_acc:.4f}")
print(f" → Best params : {best}")

results_df = pd.DataFrame(width_results)
print("\n" + "="*60)
print("Summary across all widths:")
print(results_df.to_string(index=False))

```

[I 2026-05-22 20:21:49,711] A new study created in memory with name: no-name-a643aedf-4437-43e3-a134-cb7511297df7

```

=====
Optuna study: hidden_size = 4
=====
Trial 0: acc=0.6120 | lr=1.57e-03 wd=6.35e-02 dropout=0.306 patience=20
[I 2026-05-22 20:22:16,384] Trial 0 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.001574455823978431, 'weight_decay': 0.06351221010640701, 'dropout': 0.3061978796339918, 'patience': 20}. Best is trial 0 with value: 0.6119999885559082.
Trial 1: acc=0.7540 | lr=2.10e-04 wd=4.21e-05 dropout=0.070 patience=27
[I 2026-05-22 20:23:20,480] Trial 1 finished with value: 0.7540000081062317 and parameters: {'learning_rate': 0.00021039943348033182, 'weight_decay': 4.207053950287933e-05, 'dropout': 0.07032926425886982, 'patience': 27}. Best is trial 1 with value: 0.7540000081062317.
Trial 2: acc=0.8660 | lr=1.27e-02 wd=6.80e-03 dropout=0.057 patience=30
[I 2026-05-22 20:24:21,444] Trial 2 finished with value: 0.865999966621399 and parameters: {'learning_rate': 0.012689077541328322, 'weight_decay': 0.006796578090758156, 'dropout': 0.05720457300353086, 'patience': 30}. Best is trial 2 with value: 0.865999966621399.
Trial 3: acc=0.7940 | lr=1.07e-01 wd=7.07e-05 dropout=0.114 patience=9
[I 2026-05-22 20:24:55,558] Trial 3 finished with value: 0.7940000295639038 and parameters: {'learning_rate': 0.1068416453617937, 'weight_decay': 7.068974959624602e-05, 'dropout': 0.11363873852248522, 'patience': 9}. Best is trial 2 with value: 0.865999966621399.
Trial 4: acc=0.6120 | lr=8.24e-04 wd=1.26e-03 dropout=0.201 patience=12
[I 2026-05-22 20:25:18,307] Trial 4 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.0008240223213989486, 'weight_decay': 0.0012561043700013551, 'dropout': 0.20118075652474054, 'patience': 12}. Best is trial 2 with value: 0.865999966621399.
Trial 5: acc=0.8700 | lr=1.40e-02 wd=3.61e-05 dropout=0.152 patience=14
[I 2026-05-22 20:26:29,990] Trial 5 finished with value: 0.8700000047683716 and parameters: {'learning_rate': 0.014008175793581299, 'weight_decay': 3.613894271216525e-05, 'dropout': 0.1522506269873264, 'patience': 14}. Best is trial 5 with value: 0.8700000047683716.
Trial 6: acc=0.8100 | lr=3.34e-03 wd=1.38e-02 dropout=0.120 patience=18
[I 2026-05-22 20:27:32,339] Trial 6 finished with value: 0.8100000023841858 and parameters: {'learning_rate': 0.0033361835852321023, 'weight_decay': 0.013826232179369865, 'dropout': 0.11988582375542592, 'patience': 18}. Best is trial 5 with value: 0.8700000047683716.
Trial 7: acc=0.7760 | lr=1.17e-02 wd=1.53e-05 dropout=0.263 patience=9
[I 2026-05-22 20:27:58,676] Trial 7 finished with value: 0.7760000228881836 and parameters: {'learning_rate': 0.01171192492355645, 'weight_decay': 1.5339162591163613e-05, 'dropout': 0.26264069816550345, 'patience': 9}. Best is trial 5 with value: 0.8700000047683716.
Trial 8: acc=0.6120 | lr=9.10e-05 wd=6.25e-02 dropout=0.388 patience=26
[I 2026-05-22 20:28:49,594] Trial 8 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 9.102828829203637e-05, 'weight_decay': 0.06245139574743072, 'dropout': 0.3879712115760958, 'patience': 26}. Best is trial 5 with value: 0.8700000047683716.
Trial 9: acc=0.7800 | lr=8.27e-04 wd=2.46e-05 dropout=0.289 patience=16
[I 2026-05-22 20:29:38,811] Trial 9 finished with value: 0.7799999713897705 and parameters: {'learning_rate': 0.0008268468591412214, 'weight_decay': 2.458603276328006e-05, 'dropout': 0.28948155927925495, 'patience': 16}. Best is trial 5 with value: 0.8700000047683716.
Trial 10: acc=0.6120 | lr=3.32e-01 wd=2.85e-04 dropout=0.180 patience=5
[I 2026-05-22 20:29:43,892] Trial 10 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.3324462556590268, 'weight_decay': 0.00028504320627871526, 'dropout': 0.18038819053482447, 'patience': 5}. Best is trial 5 with value: 0.8700000047683716.
Trial 11: acc=0.8820 | lr=2.26e-02 wd=2.28e-03 dropout=0.053 patience=30
[I 2026-05-22 20:30:42,155] Trial 11 finished with value: 0.8820000290870667 and parameters: {'learning_rate': 0.022634733726035822, 'weight_decay': 0.0022753430856541352, 'dropout': 0.05320709043466561, 'patience': 30}. Best is trial 11 with value: 0.8820000290870667.
Trial 12: acc=0.9040 | lr=4.40e-02 wd=5.86e-04 dropout=0.144 patience=23

```

[I 2026-05-22 20:31:53,785] Trial 12 finished with value: 0.9039999842643738 and parameters: {'learning\_rate': 0.04402019223482309, 'weight\_decay': 0.0005862766109096196, 'dropout': 0.14436134245777738, 'patience': 23}. Best is trial 12 with value: 0.9039999842643738.  
[I 2026-05-22 20:32:58,813] Trial 13 finished with value: 0.8500000238418579 and parameters: {'learning\_rate': 0.07799321598760778, 'weight\_decay': 0.0009332489055761794, 'dropout': 0.10039014245873798, 'patience': 22}. Best is trial 12 with value: 0.9039999842643738.  
Trial 13: acc=0.8500 | lr=7.80e-02 wd=9.33e-04 dropout=0.100 patience=22  
Trial 14: acc=0.8760 | lr=5.33e-02 wd=2.33e-03 dropout=0.052 patience=24  
[I 2026-05-22 20:34:08,744] Trial 14 finished with value: 0.8759999871253967 and parameters: {'learning\_rate': 0.05333422998585418, 'weight\_decay': 0.0023266686800485895, 'dropout': 0.0515274487496279, 'patience': 24}. Best is trial 12 with value: 0.9039999842643738.  
[I 2026-05-22 20:34:08,746] A new study created in memory with name: no-name-e48c6dd8-fb15-4b72-b23d-08e9f4570f07  
→ Best val accuracy : 0.9040  
→ Best params : {'learning\_rate': 0.04402019223482309, 'weight\_decay': 0.0005862766109096196, 'dropout': 0.14436134245777738, 'patience': 23}

=====  
Optuna study: hidden\_size = 8  
=====  
Trial 0: acc=0.6120 | lr=1.57e-03 wd=6.35e-02 dropout=0.306 patience=20  
[I 2026-05-22 20:34:26,147] Trial 0 finished with value: 0.6119999885559082 and parameters: {'learning\_rate': 0.001574455823978431, 'weight\_decay': 0.06351221010640701, 'dropout': 0.3061978796339918, 'patience': 20}. Best is trial 0 with value: 0.6119999885559082.  
Trial 1: acc=0.7380 | lr=2.10e-04 wd=4.21e-05 dropout=0.070 patience=27  
[I 2026-05-22 20:36:36,088] Trial 1 finished with value: 0.7379999756813049 and parameters: {'learning\_rate': 0.00021039943348033182, 'weight\_decay': 4.207053950287933e-05, 'dropout': 0.07032926425886982, 'patience': 27}. Best is trial 1 with value: 0.7379999756813049.  
Trial 2: acc=0.9020 | lr=1.27e-02 wd=6.80e-03 dropout=0.057 patience=30  
[I 2026-05-22 20:37:15,502] Trial 2 finished with value: 0.9020000100135803 and parameters: {'learning\_rate': 0.012689077541328322, 'weight\_decay': 0.006796578090758156, 'dropout': 0.05720457300353086, 'patience': 30}. Best is trial 2 with value: 0.9020000100135803.  
Trial 3: acc=0.8620 | lr=1.07e-01 wd=7.07e-05 dropout=0.114 patience=9  
[I 2026-05-22 20:37:59,146] Trial 3 finished with value: 0.8619999885559082 and parameters: {'learning\_rate': 0.1068416453617937, 'weight\_decay': 7.068974950624602e-05, 'dropout': 0.11363873852248522, 'patience': 9}. Best is trial 2 with value: 0.9020000100135803.  
Trial 4: acc=0.8080 | lr=8.24e-04 wd=1.26e-03 dropout=0.201 patience=12  
[I 2026-05-22 20:38:27,440] Trial 4 finished with value: 0.8080000281333923 and parameters: {'learning\_rate': 0.0008240223213989486, 'weight\_decay': 0.0012561043700013551, 'dropout': 0.20118075652474054, 'patience': 12}. Best is trial 2 with value: 0.9020000100135803.  
Trial 5: acc=0.7940 | lr=1.40e-02 wd=3.61e-05 dropout=0.152 patience=14  
[I 2026-05-22 20:39:14,184] Trial 5 finished with value: 0.7940000295639038 and parameters: {'learning\_rate': 0.014008175793581299, 'weight\_decay': 3.613894271216525e-05, 'dropout': 0.1522506269873264, 'patience': 14}. Best is trial 2 with value: 0.9020000100135803.  
Trial 6: acc=0.8100 | lr=3.34e-03 wd=1.38e-02 dropout=0.120 patience=18  
[I 2026-05-22 20:39:53,500] Trial 6 finished with value: 0.8100000023841858 and parameters: {'learning\_rate': 0.0033361835852321023, 'weight\_decay': 0.013826232179369865, 'dropout': 0.11988582375542592, 'patience': 18}. Best is trial 2 with value: 0.9020000100135803.  
[I 2026-05-22 20:40:30,968] Trial 7 finished with value: 0.9079999923706055 and parameters: {'learning\_rate': 0.01171192492355645, 'weight\_decay': 1.5339162591163613e-05, 'dropout': 0.26264069816550345, 'patience': 9}. Best is trial 7 with value: 0.9079999923706055.  
Trial 7: acc=0.9080 | lr=1.17e-02 wd=1.53e-05 dropout=0.263 patience=9  
[I 2026-05-22 20:40:52,132] Trial 8 finished with value: 0.6119999885559082 and parameters: {'learning\_rate': 9.102828829203637e-05, 'weight\_decay': 0.06245139574743072, 'dropout': 0.3879712115760958, 'patience': 26}. Best is trial 7 with value: 0.9079999923706055.  
Trial 8: acc=0.6120 | lr=9.10e-05 wd=6.25e-02 dropout=0.388 patience=26  
Trial 9: acc=0.7760 | lr=8.27e-04 wd=2.46e-05 dropout=0.289 patience=16  
[I 2026-05-22 20:41:32,606] Trial 9 finished with value: 0.7760000228881836 and parameters: {'learning\_rate': 0.0008268468591412214, 'weight\_decay': 2.458603276328006e-05, 'dropout': 0.28948155927925495, 'patience': 16}. Best is trial 7 with value: 0.9079999923706055.  
Trial 10: acc=0.6460 | lr=3.23e-01 wd=2.85e-04 dropout=0.253 patience=5  
[I 2026-05-22 20:41:39,299] Trial 10 finished with value: 0.6460000276565552 and parameters: {'learning\_rate': 0.3229962392989169, 'weight\_decay': 0.00028504320627871526, 'dropout': 0.2528097221376461, 'patience': 5}. Best is trial 7 with value: 0.9079999923706055.  
Trial 11: acc=0.7900 | lr=2.03e-02 wd=4.03e-03 dropout=0.201 patience=30  
[I 2026-05-22 20:42:25,407] Trial 11 finished with value: 0.7900000214576721 and parameters: {'learning\_rate': 0.020250521295708, 'weight\_decay': 0.004031500784144746, 'dropout': 0.20121274875496697, 'patience': 30}. Best is trial 7 with value: 0.9079999923706055.  
Trial 12: acc=0.9080 | lr=2.69e-02 wd=3.80e-04 dropout=0.357 patience=23  
[I 2026-05-22 20:44:11,363] Trial 12 finished with value: 0.9079999923706055 and parameters: {'learning\_rate': 0.026865656531928978, 'weight\_decay': 0.0003804703726957265, 'dropout': 0.3566035098550483, 'patience': 23}. Best is trial 7 with value: 0.9079999923706055.

Trial 13: acc=0.7920 | lr=5.96e-02 wd=2.35e-04 dropout=0.381 patience=22

[I 2026-05-22 20:44:56,710] Trial 13 finished with value: 0.7919999957084656 and parameters: {'learning\_rate': 0.05956323971846653, 'weight\_decay': 0.0002346122365666062, 'dropout': 0.3809805322269486, 'patience': 22}. Best is trial 7 with value: 0.9079999923706055.

Trial 14: acc=0.7460 | lr=7.04e-02 wd=1.18e-05 dropout=0.333 patience=23

[I 2026-05-22 20:45:30,747] Trial 14 finished with value: 0.7459999918937683 and parameters: {'learning\_rate': 0.07037178327700512, 'weight\_decay': 1.1754356101819733e-05, 'dropout': 0.33289302143468374, 'patience': 23}. Best is trial 7 with value: 0.9079999923706055.

[I 2026-05-22 20:45:30,750] A new study created in memory with name: no-name-76bade44-1aa1-4da0-96d9-e38d56be4558

→ Best val accuracy : 0.9080

→ Best params : {'learning\_rate': 0.01171192492355645, 'weight\_decay': 1.5339162591163613e-05, 'dropout': 0.26264069816550345, 'patience': 9}

=====

Optuna study: hidden\_size = 32

=====

[I 2026-05-22 20:45:48,611] Trial 0 finished with value: 0.6119999885559082 and parameters: {'learning\_rate': 0.001574455823978431, 'weight\_decay': 0.06351221010640701, 'dropout': 0.3061978796339918, 'patience': 20}. Best is trial 0 with value: 0.6119999885559082.

Trial 0: acc=0.6120 | lr=1.57e-03 wd=6.35e-02 dropout=0.306 patience=20

Trial 1: acc=0.8400 | lr=2.10e-04 wd=4.21e-05 dropout=0.070 patience=27

[I 2026-05-22 20:46:53,326] Trial 1 finished with value: 0.8399999737739563 and parameters: {'learning\_rate': 0.00021039943348033182, 'weight\_decay': 4.207053950287933e-05, 'dropout': 0.07032926425886982, 'patience': 27}. Best is trial 1 with value: 0.8399999737739563.

Trial 2: acc=0.8980 | lr=1.27e-02 wd=6.80e-03 dropout=0.057 patience=30

[I 2026-05-22 20:47:47,397] Trial 2 finished with value: 0.8980000019073486 and parameters: {'learning\_rate': 0.012689077541328322, 'weight\_decay': 0.006796578090758156, 'dropout': 0.05720457300353086, 'patience': 30}. Best is trial 2 with value: 0.8980000019073486.

Trial 3: acc=0.7820 | lr=1.07e-01 wd=7.07e-05 dropout=0.114 patience=9

[I 2026-05-22 20:48:06,291] Trial 3 finished with value: 0.7820000052452087 and parameters: {'learning\_rate': 0.1068416453617937, 'weight\_decay': 7.068974950624602e-05, 'dropout': 0.11363873852248522, 'patience': 9}. Best is trial 2 with value: 0.8980000019073486.

Trial 4: acc=0.7900 | lr=8.24e-04 wd=1.26e-03 dropout=0.201 patience=12

[I 2026-05-22 20:48:37,826] Trial 4 finished with value: 0.79000000214576721 and parameters: {'learning\_rate': 0.0008240223213989486, 'weight\_decay': 0.0012561043700013551, 'dropout': 0.20118075652474054, 'patience': 12}. Best is trial 2 with value: 0.8980000019073486.

Trial 5: acc=0.8440 | lr=1.40e-02 wd=3.61e-05 dropout=0.152 patience=14

[I 2026-05-22 20:48:58,002] Trial 5 finished with value: 0.843999981880188 and parameters: {'learning\_rate': 0.014008175793581299, 'weight\_decay': 3.613894271216525e-05, 'dropout': 0.1522506269873264, 'patience': 14}. Best is trial 2 with value: 0.8980000019073486.

Trial 6: acc=0.8240 | lr=3.34e-03 wd=1.38e-02 dropout=0.120 patience=18

[I 2026-05-22 20:49:54,834] Trial 6 finished with value: 0.8240000009536743 and parameters: {'learning\_rate': 0.0033361835852321023, 'weight\_decay': 0.013826232179369865, 'dropout': 0.11988582375542592, 'patience': 18}. Best is trial 2 with value: 0.8980000019073486.

Trial 7: acc=0.8820 | lr=1.17e-02 wd=1.53e-05 dropout=0.263 patience=9

[I 2026-05-22 20:50:32,272] Trial 7 finished with value: 0.8820000290870667 and parameters: {'learning\_rate': 0.01171192492355645, 'weight\_decay': 1.5339162591163613e-05, 'dropout': 0.26264069816550345, 'patience': 9}. Best is trial 2 with value: 0.8980000019073486.

Trial 8: acc=0.6120 | lr=9.10e-05 wd=6.25e-02 dropout=0.388 patience=26

[I 2026-05-22 20:50:58,136] Trial 8 finished with value: 0.6119999885559082 and parameters: {'learning\_rate': 9.102828829203637e-05, 'weight\_decay': 0.06245139574743072, 'dropout': 0.3879712115760958, 'patience': 26}. Best is trial 2 with value: 0.8980000019073486.

Trial 9: acc=0.7880 | lr=8.27e-04 wd=2.46e-05 dropout=0.289 patience=16

[I 2026-05-22 20:51:33,102] Trial 9 finished with value: 0.7879999876022339 and parameters: {'learning\_rate': 0.0008268468591412214, 'weight\_decay': 2.458603276328006e-05, 'dropout': 0.28948155927925495, 'patience': 16}. Best is trial 2 with value: 0.8980000019073486.

Trial 10: acc=0.6420 | lr=3.27e-01 wd=1.72e-03 dropout=0.053 patience=23

[I 2026-05-22 20:51:55,762] Trial 10 finished with value: 0.6420000195503235 and parameters: {'learning\_rate': 0.32728656656782096, 'weight\_decay': 0.0017195055939855956, 'dropout': 0.052641561760702615, 'patience': 23}. Best is trial 2 with value: 0.8980000019073486.

Trial 11: acc=0.8800 | lr=2.03e-02 wd=2.87e-04 dropout=0.242 patience=6

[I 2026-05-22 20:52:15,932] Trial 11 finished with value: 0.8799999952316284 and parameters: {'learning\_rate': 0.020250521295708, 'weight\_decay': 0.0002870122195300369, 'dropout': 0.24154374089198213, 'patience': 6}. Best is trial 2 with value: 0.8980000019073486.

Trial 12: acc=0.8020 | lr=2.69e-02 wd=7.35e-03 dropout=0.191 patience=29

[I 2026-05-22 20:52:58,714] Trial 12 finished with value: 0.8019999861717224 and parameters: {'learning\_rate': 0.026865656531928978, 'weight\_decay': 0.007346357719250234, 'dropout': 0.19106481998109492, 'patience': 29}. Best is trial 2 with value: 0.8980000019073486.

Trial 13: acc=0.8060 | lr=6.30e-02 wd=2.84e-04 dropout=0.274 patience=5



```
[I 2026-05-22 20:53:10,864] Trial 13 finished with value: 0.8059999942779541 and parameters: {'learning_rate': 0.06303393872497967, 'weight_decay': 0.00028448372828362967, 'dropout': 0.27375843249835974, 'patience': 5}. Best is trial 2 with value: 0.8980000019073486.
Trial 14: acc=0.9060 | lr=8.47e-03 wd=1.07e-05 dropout=0.351 patience=10
[I 2026-05-22 20:53:44,230] Trial 14 finished with value: 0.906000018119812 and parameters: {'learning_rate': 0.008471588991256735, 'weight_decay': 1.074688483295644e-05, 'dropout': 0.3514453873352956, 'patience': 10}. Best is trial 14 with value: 0.906000018119812.
[I 2026-05-22 20:53:44,233] A new study created in memory with name: no-name-4bf29952-beef-48b3-968f-d4d86b9e73d1
  → Best val accuracy : 0.9060
  → Best params       : {'learning_rate': 0.008471588991256735, 'weight_decay': 1.074688483295644e-05, 'dropout': 0.3514453873352956, 'patience': 10}

=====
Optuna study: hidden_size = 256
=====
Trial 0: acc=0.6120 | lr=1.57e-03 wd=6.35e-02 dropout=0.306 patience=20
[I 2026-05-22 20:54:24,777] Trial 0 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.001574455823978431, 'weight_decay': 0.06351221010640701, 'dropout': 0.3061978796339918, 'patience': 20}. Best is trial 0 with value: 0.6119999885559082.
Trial 1: acc=0.8520 | lr=2.10e-04 wd=4.21e-05 dropout=0.070 patience=27
[I 2026-05-22 20:58:06,733] Trial 1 finished with value: 0.851999980926514 and parameters: {'learning_rate': 0.00021039943348033182, 'weight_decay': 4.207053950287933e-05, 'dropout': 0.07032926425886982, 'patience': 27}. Best is trial 1 with value: 0.851999980926514.
Trial 2: acc=0.9040 | lr=1.27e-02 wd=6.80e-03 dropout=0.057 patience=30
[I 2026-05-22 21:01:25,231] Trial 2 finished with value: 0.9039999842643738 and parameters: {'learning_rate': 0.012689077541328322, 'weight_decay': 0.006796578090758156, 'dropout': 0.05720457300353086, 'patience': 30}. Best is trial 2 with value: 0.9039999842643738.
Trial 3: acc=0.6120 | lr=1.07e-01 wd=7.07e-05 dropout=0.114 patience=9
[I 2026-05-22 21:01:45,815] Trial 3 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.1068416453617937, 'weight_decay': 7.068974950624602e-05, 'dropout': 0.11363873852248522, 'patience': 9}. Best is trial 2 with value: 0.9039999842643738.
Trial 4: acc=0.9140 | lr=8.24e-04 wd=1.26e-03 dropout=0.201 patience=12
[I 2026-05-22 21:03:09,718] Trial 4 finished with value: 0.9139999747276306 and parameters: {'learning_rate': 0.0008240223213989486, 'weight_decay': 0.0012561043700013551, 'dropout': 0.20118075652474054, 'patience': 12}. Best is trial 4 with value: 0.9139999747276306.
Trial 5: acc=0.7940 | lr=1.40e-02 wd=3.61e-05 dropout=0.152 patience=14
[I 2026-05-22 21:05:54,222] Trial 5 finished with value: 0.7940000295639038 and parameters: {'learning_rate': 0.014008175793581299, 'weight_decay': 3.613894271216525e-05, 'dropout': 0.1522506269873264, 'patience': 14}. Best is trial 4 with value: 0.9139999747276306.
Trial 6: acc=0.8000 | lr=3.34e-03 wd=1.38e-02 dropout=0.120 patience=18
[I 2026-05-22 21:07:02,442] Trial 6 finished with value: 0.800000011920929 and parameters: {'learning_rate': 0.0033361835852321023, 'weight_decay': 0.013826232179369865, 'dropout': 0.11988582375542592, 'patience': 18}. Best is trial 4 with value: 0.9139999747276306.
Trial 7: acc=0.8740 | lr=1.17e-02 wd=1.53e-05 dropout=0.263 patience=9
[I 2026-05-22 21:08:02,785] Trial 7 finished with value: 0.8740000128746033 and parameters: {'learning_rate': 0.01171192492355645, 'weight_decay': 1.5339162591163613e-05, 'dropout': 0.26264069816550345, 'patience': 9}. Best is trial 4 with value: 0.9139999747276306.
Trial 8: acc=0.6120 | lr=9.10e-05 wd=6.25e-02 dropout=0.388 patience=26
[I 2026-05-22 21:08:56,487] Trial 8 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 9.102828829203637e-05, 'weight_decay': 0.06245139574743072, 'dropout': 0.3879712115760958, 'patience': 26}. Best is trial 4 with value: 0.9139999747276306.
Trial 9: acc=0.9060 | lr=8.27e-04 wd=2.46e-05 dropout=0.289 patience=16
[I 2026-05-22 21:09:58,381] Trial 9 finished with value: 0.906000018119812 and parameters: {'learning_rate': 0.0008268468591412214, 'weight_decay': 2.458603276328006e-05, 'dropout': 0.28948155927925495, 'patience': 16}. Best is trial 4 with value: 0.9139999747276306.
Trial 10: acc=0.5120 | lr=3.02e-01 wd=3.66e-04 dropout=0.200 patience=5
[I 2026-05-22 21:10:11,169] Trial 10 finished with value: 0.5120000243186951 and parameters: {'learning_rate': 0.3016589265330612, 'weight_decay': 0.0003662312545606231, 'dropout': 0.20049201947670328, 'patience': 5}. Best is trial 4 with value: 0.9139999747276306.
Trial 11: acc=0.9080 | lr=3.63e-04 wd=6.72e-04 dropout=0.310 patience=14
[I 2026-05-22 21:11:50,762] Trial 11 finished with value: 0.9079999923706055 and parameters: {'learning_rate': 0.0003627170272540882, 'weight_decay': 0.0006716180938587608, 'dropout': 0.3100820811050245, 'patience': 14}. Best is trial 4 with value: 0.9139999747276306.
Trial 12: acc=0.8980 | lr=4.45e-04 wd=8.05e-04 dropout=0.355 patience=12
[I 2026-05-22 21:12:54,688] Trial 12 finished with value: 0.8980000019073486 and parameters: {'learning_rate': 0.0004451286014346576, 'weight_decay': 0.0008054255643182018, 'dropout': 0.35539519787125473, 'patience': 12}. Best is trial 4 with value: 0.9139999747276306.
Trial 13: acc=0.8840 | lr=7.33e-05 wd=2.76e-03 dropout=0.215 patience=21
```

```
[I 2026-05-22 21:17:25,154] Trial 13 finished with value: 0.8840000033378601 and parameters: {'learning_rate': 7.326450782462902e-05, 'weight_decay': 0.0027594931247927014, 'dropout': 0.21544990233028208, 'patience': 21}. Best is trial 4 with value: 0.9139999747276306.
Trial 14: acc=0.8760 | lr=2.18e-04 wd=2.02e-04 dropout=0.325 patience=10
[I 2026-05-22 21:18:46,062] Trial 14 finished with value: 0.8759999871253967 and parameters: {'learning_rate': 0.00021836618979918725, 'weight_decay': 0.0002016856717436378, 'dropout': 0.3248437915678488, 'patience': 10}. Best is trial 4 with value: 0.9139999747276306.
[I 2026-05-22 21:18:46,066] A new study created in memory with name: no-name-644866a4-caa9-4ee2-8ff3-465d3678f2de
  → Best val accuracy : 0.9140
  → Best params       : {'learning_rate': 0.0008240223213989486, 'weight_decay': 0.0012561043700013551, 'dropout': 0.20118075652474054, 'patience': 12}

=====
Optuna study: hidden_size = 1028
=====
Trial 0: acc=0.6120 | lr=1.57e-03 wd=6.35e-02 dropout=0.306 patience=20
[I 2026-05-22 21:25:23,116] Trial 0 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.001574455823978431, 'weight_decay': 0.06351221010640701, 'dropout': 0.3061978796339918, 'patience': 20}. Best is trial 0 with value: 0.6119999885559082.
Trial 1: acc=0.8700 | lr=2.10e-04 wd=4.21e-05 dropout=0.070 patience=27
[I 2026-05-22 21:40:16,962] Trial 1 finished with value: 0.8700000047683716 and parameters: {'learning_rate': 0.00021039943348033182, 'weight_decay': 4.207053950287933e-05, 'dropout': 0.07032926425886982, 'patience': 27}. Best is trial 1 with value: 0.8700000047683716.
Trial 2: acc=0.7940 | lr=1.27e-02 wd=6.80e-03 dropout=0.057 patience=30
[I 2026-05-22 22:07:24,671] Trial 2 finished with value: 0.7940000295639038 and parameters: {'learning_rate': 0.012689077541328322, 'weight_decay': 0.006796578090758156, 'dropout': 0.05720457300353086, 'patience': 30}. Best is trial 1 with value: 0.8700000047683716.
Trial 3: acc=0.6120 | lr=1.07e-01 wd=7.07e-05 dropout=0.114 patience=9
[I 2026-05-22 22:10:27,390] Trial 3 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.1068416453617937, 'weight_decay': 7.068974950624602e-05, 'dropout': 0.11363873852248522, 'patience': 9}. Best is trial 1 with value: 0.8700000047683716.
Trial 4: acc=0.9140 | lr=8.24e-04 wd=1.26e-03 dropout=0.201 patience=12
[I 2026-05-22 22:28:22,185] Trial 4 finished with value: 0.9139999747276306 and parameters: {'learning_rate': 0.0008240223213989486, 'weight_decay': 0.0012561043700013551, 'dropout': 0.20118075652474054, 'patience': 12}. Best is trial 4 with value: 0.9139999747276306.
Trial 5: acc=0.8060 | lr=1.40e-02 wd=3.61e-05 dropout=0.152 patience=14
[I 2026-05-22 22:54:48,836] Trial 5 finished with value: 0.8059999942779541 and parameters: {'learning_rate': 0.014008175793581299, 'weight_decay': 3.613894271216525e-05, 'dropout': 0.1522506269873264, 'patience': 14}. Best is trial 4 with value: 0.9139999747276306.
Trial 6: acc=0.8260 | lr=3.34e-03 wd=1.38e-02 dropout=0.120 patience=18
[I 2026-05-22 23:07:56,123] Trial 6 finished with value: 0.8259999752044678 and parameters: {'learning_rate': 0.0033361835852321023, 'weight_decay': 0.013826232179369865, 'dropout': 0.11988582375542592, 'patience': 18}. Best is trial 4 with value: 0.9139999747276306.
Trial 7: acc=0.6120 | lr=1.17e-02 wd=1.53e-05 dropout=0.263 patience=9
[I 2026-05-22 23:12:06,940] Trial 7 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 0.01171192492355645, 'weight_decay': 1.5339162591163613e-05, 'dropout': 0.26264069816550345, 'patience': 9}. Best is trial 4 with value: 0.9139999747276306.
Trial 8: acc=0.6120 | lr=9.10e-05 wd=6.25e-02 dropout=0.388 patience=26
[I 2026-05-22 23:22:54,338] Trial 8 finished with value: 0.6119999885559082 and parameters: {'learning_rate': 9.102828829203637e-05, 'weight_decay': 0.06245139574743072, 'dropout': 0.3879712115760958, 'patience': 26}. Best is trial 4 with value: 0.9139999747276306.
Trial 9: acc=0.8880 | lr=8.27e-04 wd=2.46e-05 dropout=0.289 patience=16
[I 2026-05-22 23:49:58,550] Trial 9 finished with value: 0.8880000114440918 and parameters: {'learning_rate': 0.0008268468591412214, 'weight_decay': 2.458603276328006e-05, 'dropout': 0.28948155927925495, 'patience': 16}. Best is trial 4 with value: 0.9139999747276306.
Trial 10: acc=0.5580 | lr=3.02e-01 wd=3.66e-04 dropout=0.200 patience=5
[I 2026-05-22 23:55:45,036] Trial 10 finished with value: 0.5580000281333923 and parameters: {'learning_rate': 0.3016589265330612, 'weight_decay': 0.0003662312545606231, 'dropout': 0.20049201947670328, 'patience': 5}. Best is trial 4 with value: 0.9139999747276306.
Trial 11: acc=0.9160 | lr=3.63e-04 wd=6.72e-04 dropout=0.310 patience=14
[I 2026-05-23 00:16:35,406] Trial 11 finished with value: 0.9160000085830688 and parameters: {'learning_rate': 0.0003627170272540882, 'weight_decay': 0.0006716180938587608, 'dropout': 0.3100820811050245, 'patience': 14}. Best is trial 11 with value: 0.9160000085830688.
Trial 12: acc=0.9020 | lr=4.45e-04 wd=8.05e-04 dropout=0.355 patience=12
[I 2026-05-23 00:31:52,835] Trial 12 finished with value: 0.9020000100135803 and parameters: {'learning_rate': 0.0004451286014346576, 'weight_decay': 0.0008054255643182018, 'dropout': 0.35539519787125473, 'patience': 12}. Best is trial 11 with value: 0.9160000085830688.
Trial 13: acc=0.8880 | lr=7.33e-05 wd=2.76e-03 dropout=0.215 patience=21
```

```
[I 2026-05-23 00:57:47,316] Trial 13 finished with value: 0.8880000114440918 and parameters: {'learning_rate': 7.326450782462902e-05, 'weight_decay': 0.0027594931247927014, 'dropout': 0.21544990233028208, 'patience': 21}. Best is trial 11 with value: 0.9160000085830688.
Trial 14: acc=0.9060 | lr=2.18e-04 wd=2.02e-04 dropout=0.325 patience=10
[I 2026-05-23 01:06:05,918] Trial 14 finished with value: 0.906000018119812 and parameters: {'learning_rate': 0.00021836618979918725, 'weight_decay': 0.0002016856717436378, 'dropout': 0.3248437915678488, 'patience': 10}. Best is trial 11 with value: 0.9160000085830688.
→ Best val accuracy : 0.9160
→ Best params      : {'learning_rate': 0.0003627170272540882, 'weight_decay': 0.0006716180938587608, 'dropout': 0.3100820811050245, 'patience': 14}
```

Summary across all widths:

| hidden_size | best_val_acc | learning_rate | weight_decay | dropout  | patience |
|-------------|--------------|---------------|--------------|----------|----------|
| 4           | 0.904        | 0.044020      | 0.000586     | 0.144361 | 23       |
| 8           | 0.908        | 0.011712      | 0.000015     | 0.262641 | 9        |
| 32          | 0.906        | 0.008472      | 0.000011     | 0.351445 | 10       |
| 256         | 0.914        | 0.000824      | 0.001256     | 0.201181 | 12       |
| 1028        | 0.916        | 0.000363      | 0.000672     | 0.310082 | 14       |

## Retrain Best Model per Width and Evaluate on Held-Out Test Set

Optuna's internal accuracy was measured on the validation set. We now retrain the best configuration for each width and evaluate on the truly held-out test set.

```
In [9]: final_test_results = []

for row in width_results:
    w      = row["hidden_size"]
    lr     = row["learning_rate"]
    wd     = row["weight_decay"]
    drop   = row["dropout"]
    patience = row["patience"]

    print(f"\nRetraining best model for width={w} (lr={lr:.2e}, wd={wd:.2e}, drop={drop:.3f})")

    model      = LSTMPreEmbedded(hidden_size=w, num_layers=NUM_LAYERS, dropout=drop)
    optimizer  = Adam(model.parameters(), lr=lr, weight_decay=wd)
    module     = SimpleModule.classification(model, optimizer=optimizer, num_classes=3)

    ckpt_dir  = f'ckpt/best_w{w}'
    shutil.rmtree(ckpt_dir, ignore_errors=True)

    checkpoint_cb = ModelCheckpoint(
        monitor='valid_loss', mode='min', save_top_k=1, dirpath=ckpt_dir)
    early_stop_cb = EarlyStopping(
        monitor='valid_accuracy', mode='max', patience=patience)
    logger = CSVLogger('logs', name=f'LSTM_w{w}')

    trainer = Trainer(
        deterministic=True,
        max_epochs=200,
        logger=logger,
        enable_progress_bar=False,
        enable_model_summary=False,
        callbacks=[ErrorTracker(), PrintMetrics(), checkpoint_cb, early_stop_cb]
    )

    trainer.fit(module, datamodule=seq_dm_emb)
    test_out = trainer.test(module, datamodule=seq_dm_emb, ckpt_path='best', verbose=False)
    test_acc = test_out[0]['test_accuracy']

    print(f" → Final test accuracy: {test_acc:.4f}")

    final_test_results.append({
        "hidden_size": w,
        "test_accuracy": test_acc,
    })

shutil.rmtree(ckpt_dir, ignore_errors=True)
del model, module, trainer
gc.collect()
```

```

torch.cuda.empty_cache()

final_df = pd.DataFrame(final_test_results)
print("\n" + "="*60)
print("Final test accuracy by width:")
print(final_df[["hidden_size", "best_val_acc", "test_accuracy"]].to_string(index=False))

```

Retraining best model for width=4 (lr=4.40e-02, wd=5.86e-04, drop=0.144)

|       |    |                    |                   |                    |                   |
|-------|----|--------------------|-------------------|--------------------|-------------------|
| Epoch | 1  | train_loss: 0.8831 | train_acc: 0.7500 | valid_loss: 0.6282 | valid_acc: 0.7360 |
| Epoch | 2  | train_loss: 0.6959 | train_acc: 0.7083 | valid_loss: 0.6181 | valid_acc: 0.7260 |
| Epoch | 3  | train_loss: 0.5977 | train_acc: 0.6458 | valid_loss: 0.6148 | valid_acc: 0.7200 |
| Epoch | 4  | train_loss: 0.5726 | train_acc: 0.7917 | valid_loss: 0.7505 | valid_acc: 0.7360 |
| Epoch | 5  | train_loss: 0.5103 | train_acc: 0.7708 | valid_loss: 0.5286 | valid_acc: 0.7700 |
| Epoch | 6  | train_loss: 0.5096 | train_acc: 0.7708 | valid_loss: 0.5628 | valid_acc: 0.7620 |
| Epoch | 7  | train_loss: 0.4195 | train_acc: 0.8125 | valid_loss: 0.6143 | valid_acc: 0.7560 |
| Epoch | 8  | train_loss: 0.4485 | train_acc: 0.7708 | valid_loss: 0.4719 | valid_acc: 0.7980 |
| Epoch | 9  | train_loss: 0.4268 | train_acc: 0.7292 | valid_loss: 0.5558 | valid_acc: 0.7600 |
| Epoch | 10 | train_loss: 0.3865 | train_acc: 0.7708 | valid_loss: 0.4455 | valid_acc: 0.7880 |
| Epoch | 11 | train_loss: 0.4318 | train_acc: 0.6042 | valid_loss: 0.5203 | valid_acc: 0.7800 |
| Epoch | 12 | train_loss: 0.4737 | train_acc: 0.7917 | valid_loss: 0.4832 | valid_acc: 0.8080 |
| Epoch | 13 | train_loss: 0.5050 | train_acc: 0.8125 | valid_loss: 0.4994 | valid_acc: 0.8000 |
| Epoch | 14 | train_loss: 0.3982 | train_acc: 0.8542 | valid_loss: 0.4690 | valid_acc: 0.8080 |
| Epoch | 15 | train_loss: 0.3874 | train_acc: 0.8542 | valid_loss: 0.5035 | valid_acc: 0.8020 |
| Epoch | 16 | train_loss: 0.3892 | train_acc: 0.8333 | valid_loss: 0.4616 | valid_acc: 0.8060 |
| Epoch | 17 | train_loss: 0.4190 | train_acc: 0.8542 | valid_loss: 0.5119 | valid_acc: 0.7920 |
| Epoch | 18 | train_loss: 0.3573 | train_acc: 0.7917 | valid_loss: 0.4836 | valid_acc: 0.7940 |
| Epoch | 19 | train_loss: 0.3683 | train_acc: 0.8125 | valid_loss: 0.5224 | valid_acc: 0.7980 |
| Epoch | 20 | train_loss: 0.3974 | train_acc: 0.7917 | valid_loss: 0.5338 | valid_acc: 0.7820 |
| Epoch | 21 | train_loss: 0.3983 | train_acc: 0.7917 | valid_loss: 0.5182 | valid_acc: 0.7980 |
| Epoch | 22 | train_loss: 0.4105 | train_acc: 0.8333 | valid_loss: 0.4859 | valid_acc: 0.8040 |
| Epoch | 23 | train_loss: 0.3623 | train_acc: 0.6042 | valid_loss: 0.4813 | valid_acc: 0.7600 |
| Epoch | 24 | train_loss: 0.3620 | train_acc: 0.8333 | valid_loss: 0.4684 | valid_acc: 0.7880 |
| Epoch | 25 | train_loss: 0.3780 | train_acc: 0.8125 | valid_loss: 0.4802 | valid_acc: 0.7880 |
| Epoch | 26 | train_loss: 0.4198 | train_acc: 0.8125 | valid_loss: 0.5123 | valid_acc: 0.7620 |
| Epoch | 27 | train_loss: 0.3814 | train_acc: 0.8542 | valid_loss: 0.4630 | valid_acc: 0.8060 |
| Epoch | 28 | train_loss: 0.3665 | train_acc: 0.8125 | valid_loss: 0.4800 | valid_acc: 0.7920 |
| Epoch | 29 | train_loss: 0.3600 | train_acc: 0.7708 | valid_loss: 0.5323 | valid_acc: 0.7760 |
| Epoch | 30 | train_loss: 0.3516 | train_acc: 0.8125 | valid_loss: 0.5154 | valid_acc: 0.7720 |
| Epoch | 31 | train_loss: 0.3830 | train_acc: 0.8750 | valid_loss: 0.5008 | valid_acc: 0.7840 |
| Epoch | 32 | train_loss: 0.3732 | train_acc: 0.7500 | valid_loss: 0.4620 | valid_acc: 0.7800 |
| Epoch | 33 | train_loss: 0.3618 | train_acc: 0.9167 | valid_loss: 0.4150 | valid_acc: 0.8360 |
| Epoch | 34 | train_loss: 0.3568 | train_acc: 0.9583 | valid_loss: 0.3895 | valid_acc: 0.8680 |
| Epoch | 35 | train_loss: 0.3231 | train_acc: 0.8542 | valid_loss: 0.3807 | valid_acc: 0.8920 |
| Epoch | 36 | train_loss: 0.2800 | train_acc: 0.9167 | valid_loss: 0.4255 | valid_acc: 0.8460 |
| Epoch | 37 | train_loss: 0.2863 | train_acc: 0.8333 | valid_loss: 0.4586 | valid_acc: 0.8420 |
| Epoch | 38 | train_loss: 0.3518 | train_acc: 0.9167 | valid_loss: 0.4385 | valid_acc: 0.8720 |
| Epoch | 39 | train_loss: 0.3674 | train_acc: 0.8542 | valid_loss: 0.3952 | valid_acc: 0.8820 |
| Epoch | 40 | train_loss: 0.3074 | train_acc: 0.8750 | valid_loss: 0.3740 | valid_acc: 0.9020 |
| Epoch | 41 | train_loss: 0.2726 | train_acc: 0.8333 | valid_loss: 0.3969 | valid_acc: 0.8660 |
| Epoch | 42 | train_loss: 0.3081 | train_acc: 0.8333 | valid_loss: 0.3708 | valid_acc: 0.8720 |
| Epoch | 43 | train_loss: 0.3130 | train_acc: 0.9375 | valid_loss: 0.3665 | valid_acc: 0.8720 |
| Epoch | 44 | train_loss: 0.2795 | train_acc: 0.7708 | valid_loss: 0.3670 | valid_acc: 0.8720 |
| Epoch | 45 | train_loss: 0.2779 | train_acc: 0.9583 | valid_loss: 0.4218 | valid_acc: 0.8820 |
| Epoch | 46 | train_loss: 0.3059 | train_acc: 0.8750 | valid_loss: 0.3463 | valid_acc: 0.8880 |
| Epoch | 47 | train_loss: 0.2856 | train_acc: 0.8333 | valid_loss: 0.3689 | valid_acc: 0.8900 |
| Epoch | 48 | train_loss: 0.2453 | train_acc: 0.8542 | valid_loss: 0.3430 | valid_acc: 0.9000 |
| Epoch | 49 | train_loss: 0.2774 | train_acc: 0.9583 | valid_loss: 0.3416 | valid_acc: 0.8920 |
| Epoch | 50 | train_loss: 0.2495 | train_acc: 0.9167 | valid_loss: 0.3523 | valid_acc: 0.8880 |
| Epoch | 51 | train_loss: 0.2511 | train_acc: 0.8542 | valid_loss: 0.6249 | valid_acc: 0.8440 |
| Epoch | 52 | train_loss: 0.2442 | train_acc: 0.8750 | valid_loss: 0.2968 | valid_acc: 0.8900 |
| Epoch | 53 | train_loss: 0.2759 | train_acc: 0.9792 | valid_loss: 0.3627 | valid_acc: 0.8820 |
| Epoch | 54 | train_loss: 0.2122 | train_acc: 0.8958 | valid_loss: 0.3250 | valid_acc: 0.8820 |
| Epoch | 55 | train_loss: 0.2199 | train_acc: 0.9167 | valid_loss: 0.3514 | valid_acc: 0.8960 |
| Epoch | 56 | train_loss: 0.2402 | train_acc: 0.8333 | valid_loss: 0.3200 | valid_acc: 0.8840 |
| Epoch | 57 | train_loss: 0.2685 | train_acc: 0.9167 | valid_loss: 0.3497 | valid_acc: 0.8880 |
| Epoch | 58 | train_loss: 0.2573 | train_acc: 0.8333 | valid_loss: 0.3272 | valid_acc: 0.8760 |
| Epoch | 59 | train_loss: 0.2775 | train_acc: 0.9167 | valid_loss: 0.3467 | valid_acc: 0.8960 |
| Epoch | 60 | train_loss: 0.2665 | train_acc: 0.8750 | valid_loss: 0.3774 | valid_acc: 0.8880 |
| Epoch | 61 | train_loss: 0.2401 | train_acc: 0.8750 | valid_loss: 0.3071 | valid_acc: 0.9040 |
| Epoch | 62 | train_loss: 0.2386 | train_acc: 0.8958 | valid_loss: 0.3782 | valid_acc: 0.8940 |
| Epoch | 63 | train_loss: 0.2306 | train_acc: 0.9375 | valid_loss: 0.4320 | valid_acc: 0.8700 |
| Epoch | 64 | train_loss: 0.2122 | train_acc: 0.8542 | valid_loss: 0.3412 | valid_acc: 0.8980 |
| Epoch | 65 | train_loss: 0.2518 | train_acc: 0.9167 | valid_loss: 0.3503 | valid_acc: 0.8920 |
| Epoch | 66 | train_loss: 0.2498 | train_acc: 0.9583 | valid_loss: 0.2908 | valid_acc: 0.9020 |
| Epoch | 67 | train_loss: 0.2602 | train_acc: 0.9375 | valid_loss: 0.2831 | valid_acc: 0.8960 |
| Epoch | 68 | train_loss: 0.2178 | train_acc: 0.8958 | valid_loss: 0.3893 | valid_acc: 0.8860 |
| Epoch | 69 | train_loss: 0.2093 | train_acc: 0.8958 | valid_loss: 0.4095 | valid_acc: 0.8640 |

|       |    |  |                    |  |                   |  |                    |  |                   |
|-------|----|--|--------------------|--|-------------------|--|--------------------|--|-------------------|
| Epoch | 70 |  | train_loss: 0.1972 |  | train_acc: 0.8958 |  | valid_loss: 0.2851 |  | valid_acc: 0.9120 |
| Epoch | 71 |  | train_loss: 0.2509 |  | train_acc: 0.8750 |  | valid_loss: 0.3834 |  | valid_acc: 0.8920 |
| Epoch | 72 |  | train_loss: 0.2174 |  | train_acc: 0.9792 |  | valid_loss: 0.3587 |  | valid_acc: 0.8840 |
| Epoch | 73 |  | train_loss: 0.2435 |  | train_acc: 0.9167 |  | valid_loss: 0.3284 |  | valid_acc: 0.9040 |
| Epoch | 74 |  | train_loss: 0.2247 |  | train_acc: 0.9167 |  | valid_loss: 0.4098 |  | valid_acc: 0.8720 |
| Epoch | 75 |  | train_loss: 0.2309 |  | train_acc: 0.8750 |  | valid_loss: 0.2950 |  | valid_acc: 0.9000 |
| Epoch | 76 |  | train_loss: 0.2653 |  | train_acc: 0.9167 |  | valid_loss: 0.2870 |  | valid_acc: 0.9080 |
| Epoch | 77 |  | train_loss: 0.2159 |  | train_acc: 0.9375 |  | valid_loss: 0.3034 |  | valid_acc: 0.8980 |
| Epoch | 78 |  | train_loss: 0.2172 |  | train_acc: 0.8958 |  | valid_loss: 0.3315 |  | valid_acc: 0.8940 |
| Epoch | 79 |  | train_loss: 0.2227 |  | train_acc: 0.9583 |  | valid_loss: 0.3858 |  | valid_acc: 0.8780 |
| Epoch | 80 |  | train_loss: 0.2382 |  | train_acc: 0.9583 |  | valid_loss: 0.3449 |  | valid_acc: 0.8920 |
| Epoch | 81 |  | train_loss: 0.2209 |  | train_acc: 0.8750 |  | valid_loss: 0.3586 |  | valid_acc: 0.8740 |
| Epoch | 82 |  | train_loss: 0.2122 |  | train_acc: 0.9375 |  | valid_loss: 0.3284 |  | valid_acc: 0.8940 |
| Epoch | 83 |  | train_loss: 0.2457 |  | train_acc: 0.8125 |  | valid_loss: 0.3395 |  | valid_acc: 0.9040 |
| Epoch | 84 |  | train_loss: 0.2298 |  | train_acc: 0.9375 |  | valid_loss: 0.3621 |  | valid_acc: 0.8980 |
| Epoch | 85 |  | train_loss: 0.2028 |  | train_acc: 0.9583 |  | valid_loss: 0.4096 |  | valid_acc: 0.8640 |
| Epoch | 86 |  | train_loss: 0.2840 |  | train_acc: 0.8958 |  | valid_loss: 0.3087 |  | valid_acc: 0.9000 |
| Epoch | 87 |  | train_loss: 0.2369 |  | train_acc: 0.9375 |  | valid_loss: 0.2771 |  | valid_acc: 0.9020 |
| Epoch | 88 |  | train_loss: 0.2231 |  | train_acc: 0.9375 |  | valid_loss: 0.3028 |  | valid_acc: 0.8940 |
| Epoch | 89 |  | train_loss: 0.2666 |  | train_acc: 0.9375 |  | valid_loss: 0.2902 |  | valid_acc: 0.9040 |
| Epoch | 90 |  | train_loss: 0.2145 |  | train_acc: 0.9792 |  | valid_loss: 0.2729 |  | valid_acc: 0.9060 |
| Epoch | 91 |  | train_loss: 0.2264 |  | train_acc: 0.9375 |  | valid_loss: 0.3062 |  | valid_acc: 0.8960 |
| Epoch | 92 |  | train_loss: 0.1839 |  | train_acc: 0.9583 |  | valid_loss: 0.2765 |  | valid_acc: 0.9000 |
| Epoch | 93 |  | train_loss: 0.1937 |  | train_acc: 0.8750 |  | valid_loss: 0.2866 |  | valid_acc: 0.8940 |

→ Final test accuracy: 0.9020

Retraining best model for width=8 (lr=1.17e-02, wd=1.53e-05, drop=0.263)

|       |    |  |                    |  |                   |  |                    |  |                   |
|-------|----|--|--------------------|--|-------------------|--|--------------------|--|-------------------|
| Epoch | 1  |  | train_loss: 0.9336 |  | train_acc: 0.7292 |  | valid_loss: 0.6588 |  | valid_acc: 0.7260 |
| Epoch | 2  |  | train_loss: 0.7345 |  | train_acc: 0.6458 |  | valid_loss: 0.6367 |  | valid_acc: 0.7420 |
| Epoch | 3  |  | train_loss: 0.6272 |  | train_acc: 0.7500 |  | valid_loss: 0.6253 |  | valid_acc: 0.7400 |
| Epoch | 4  |  | train_loss: 0.5807 |  | train_acc: 0.7708 |  | valid_loss: 0.5800 |  | valid_acc: 0.7680 |
| Epoch | 5  |  | train_loss: 0.5619 |  | train_acc: 0.7917 |  | valid_loss: 0.5443 |  | valid_acc: 0.7780 |
| Epoch | 6  |  | train_loss: 0.4795 |  | train_acc: 0.8750 |  | valid_loss: 0.5386 |  | valid_acc: 0.7900 |
| Epoch | 7  |  | train_loss: 0.4525 |  | train_acc: 0.8750 |  | valid_loss: 0.5136 |  | valid_acc: 0.7840 |
| Epoch | 8  |  | train_loss: 0.4492 |  | train_acc: 0.8125 |  | valid_loss: 0.5270 |  | valid_acc: 0.7880 |
| Epoch | 9  |  | train_loss: 0.4126 |  | train_acc: 0.8125 |  | valid_loss: 0.5006 |  | valid_acc: 0.8020 |
| Epoch | 10 |  | train_loss: 0.3890 |  | train_acc: 0.8333 |  | valid_loss: 0.5566 |  | valid_acc: 0.7940 |
| Epoch | 11 |  | train_loss: 0.3705 |  | train_acc: 0.8750 |  | valid_loss: 0.5405 |  | valid_acc: 0.8000 |
| Epoch | 12 |  | train_loss: 0.3477 |  | train_acc: 0.8750 |  | valid_loss: 0.5222 |  | valid_acc: 0.8060 |
| Epoch | 13 |  | train_loss: 0.3525 |  | train_acc: 0.7708 |  | valid_loss: 0.5231 |  | valid_acc: 0.8040 |
| Epoch | 14 |  | train_loss: 0.3526 |  | train_acc: 0.8125 |  | valid_loss: 0.4985 |  | valid_acc: 0.8100 |
| Epoch | 15 |  | train_loss: 0.3290 |  | train_acc: 0.8542 |  | valid_loss: 0.5633 |  | valid_acc: 0.8040 |
| Epoch | 16 |  | train_loss: 0.3240 |  | train_acc: 0.7917 |  | valid_loss: 0.5516 |  | valid_acc: 0.7960 |
| Epoch | 17 |  | train_loss: 0.3288 |  | train_acc: 0.8333 |  | valid_loss: 0.5485 |  | valid_acc: 0.8000 |
| Epoch | 18 |  | train_loss: 0.3131 |  | train_acc: 0.9167 |  | valid_loss: 0.5512 |  | valid_acc: 0.7920 |
| Epoch | 19 |  | train_loss: 0.3150 |  | train_acc: 0.8542 |  | valid_loss: 0.6197 |  | valid_acc: 0.7980 |
| Epoch | 20 |  | train_loss: 0.3047 |  | train_acc: 0.8542 |  | valid_loss: 0.5255 |  | valid_acc: 0.7940 |
| Epoch | 21 |  | train_loss: 0.3004 |  | train_acc: 0.8750 |  | valid_loss: 0.5127 |  | valid_acc: 0.7820 |
| Epoch | 22 |  | train_loss: 0.3030 |  | train_acc: 0.8333 |  | valid_loss: 0.5362 |  | valid_acc: 0.8060 |
| Epoch | 23 |  | train_loss: 0.2835 |  | train_acc: 0.8750 |  | valid_loss: 0.5359 |  | valid_acc: 0.8060 |

→ Final test accuracy: 0.7840

Retraining best model for width=32 (lr=8.47e-03, wd=1.07e-05, drop=0.351)

|       |    |  |                    |  |                   |  |                    |  |                   |
|-------|----|--|--------------------|--|-------------------|--|--------------------|--|-------------------|
| Epoch | 1  |  | train_loss: 0.8609 |  | train_acc: 0.7083 |  | valid_loss: 0.5886 |  | valid_acc: 0.7540 |
| Epoch | 2  |  | train_loss: 0.6475 |  | train_acc: 0.8542 |  | valid_loss: 0.5707 |  | valid_acc: 0.7600 |
| Epoch | 3  |  | train_loss: 0.5523 |  | train_acc: 0.8750 |  | valid_loss: 0.5278 |  | valid_acc: 0.7740 |
| Epoch | 4  |  | train_loss: 0.4630 |  | train_acc: 0.8333 |  | valid_loss: 0.5128 |  | valid_acc: 0.7960 |
| Epoch | 5  |  | train_loss: 0.4086 |  | train_acc: 0.8125 |  | valid_loss: 0.5042 |  | valid_acc: 0.7980 |
| Epoch | 6  |  | train_loss: 0.3495 |  | train_acc: 0.8958 |  | valid_loss: 0.5660 |  | valid_acc: 0.8180 |
| Epoch | 7  |  | train_loss: 0.2805 |  | train_acc: 0.9583 |  | valid_loss: 0.4717 |  | valid_acc: 0.8700 |
| Epoch | 8  |  | train_loss: 0.2356 |  | train_acc: 0.8750 |  | valid_loss: 0.6115 |  | valid_acc: 0.8420 |
| Epoch | 9  |  | train_loss: 0.1900 |  | train_acc: 0.9583 |  | valid_loss: 0.5494 |  | valid_acc: 0.8640 |
| Epoch | 10 |  | train_loss: 0.2015 |  | train_acc: 0.9375 |  | valid_loss: 0.5475 |  | valid_acc: 0.8500 |
| Epoch | 11 |  | train_loss: 0.1941 |  | train_acc: 0.8542 |  | valid_loss: 0.5550 |  | valid_acc: 0.8660 |
| Epoch | 12 |  | train_loss: 0.1835 |  | train_acc: 0.9375 |  | valid_loss: 0.4957 |  | valid_acc: 0.8740 |
| Epoch | 13 |  | train_loss: 0.1382 |  | train_acc: 0.8750 |  | valid_loss: 0.5070 |  | valid_acc: 0.8780 |
| Epoch | 14 |  | train_loss: 0.1275 |  | train_acc: 0.9167 |  | valid_loss: 0.3828 |  | valid_acc: 0.8860 |
| Epoch | 15 |  | train_loss: 0.1589 |  | train_acc: 1.0000 |  | valid_loss: 0.4169 |  | valid_acc: 0.8900 |
| Epoch | 16 |  | train_loss: 0.1447 |  | train_acc: 0.9792 |  | valid_loss: 0.4416 |  | valid_acc: 0.9000 |
| Epoch | 17 |  | train_loss: 0.1189 |  | train_acc: 0.9583 |  | valid_loss: 0.5354 |  | valid_acc: 0.8960 |
| Epoch | 18 |  | train_loss: 0.1080 |  | train_acc: 1.0000 |  | valid_loss: 0.4647 |  | valid_acc: 0.8860 |
| Epoch | 19 |  | train_loss: 0.0977 |  | train_acc: 1.0000 |  | valid_loss: 0.4764 |  | valid_acc: 0.8920 |
| Epoch | 20 |  | train_loss: 0.0968 |  | train_acc: 0.9375 |  | valid_loss: 0.4638 |  | valid_acc: 0.8940 |
| Epoch | 21 |  | train_loss: 0.0836 |  | train_acc: 0.9792 |  | valid_loss: 0.3812 |  | valid_acc: 0.8860 |
| Epoch | 22 |  | train_loss: 0.0956 |  | train_acc: 0.9792 |  | valid_loss: 0.4401 |  | valid_acc: 0.9080 |
| Epoch | 23 |  | train_loss: 0.0653 |  | train_acc: 1.0000 |  | valid_loss: 0.3981 |  | valid_acc: 0.9060 |
| Epoch | 24 |  | train_loss: 0.0900 |  | train_acc: 0.9792 |  | valid_loss: 0.4500 |  | valid_acc: 0.9140 |

|       |    |                    |                   |                    |                   |
|-------|----|--------------------|-------------------|--------------------|-------------------|
| Epoch | 25 | train_loss: 0.0736 | train_acc: 1.0000 | valid_loss: 0.4673 | valid_acc: 0.9080 |
| Epoch | 26 | train_loss: 0.0671 | train_acc: 1.0000 | valid_loss: 0.4760 | valid_acc: 0.9020 |
| Epoch | 27 | train_loss: 0.0545 | train_acc: 0.9375 | valid_loss: 0.4395 | valid_acc: 0.9020 |
| Epoch | 28 | train_loss: 0.0523 | train_acc: 0.9583 | valid_loss: 0.4625 | valid_acc: 0.9140 |
| Epoch | 29 | train_loss: 0.0519 | train_acc: 0.9792 | valid_loss: 0.5023 | valid_acc: 0.9060 |
| Epoch | 30 | train_loss: 0.0512 | train_acc: 1.0000 | valid_loss: 0.5321 | valid_acc: 0.9240 |
| Epoch | 31 | train_loss: 0.0450 | train_acc: 0.9792 | valid_loss: 0.5726 | valid_acc: 0.9120 |
| Epoch | 32 | train_loss: 0.0455 | train_acc: 1.0000 | valid_loss: 0.6264 | valid_acc: 0.8900 |
| Epoch | 33 | train_loss: 0.0499 | train_acc: 1.0000 | valid_loss: 0.4344 | valid_acc: 0.9120 |
| Epoch | 34 | train_loss: 0.0829 | train_acc: 0.9792 | valid_loss: 0.3770 | valid_acc: 0.9160 |
| Epoch | 35 | train_loss: 0.0700 | train_acc: 0.9792 | valid_loss: 0.4827 | valid_acc: 0.9100 |
| Epoch | 36 | train_loss: 0.0643 | train_acc: 1.0000 | valid_loss: 0.4022 | valid_acc: 0.9260 |
| Epoch | 37 | train_loss: 0.0700 | train_acc: 0.9792 | valid_loss: 0.3958 | valid_acc: 0.9120 |
| Epoch | 38 | train_loss: 0.0486 | train_acc: 1.0000 | valid_loss: 0.3976 | valid_acc: 0.9260 |
| Epoch | 39 | train_loss: 0.0926 | train_acc: 1.0000 | valid_loss: 0.4122 | valid_acc: 0.9160 |
| Epoch | 40 | train_loss: 0.0597 | train_acc: 0.9375 | valid_loss: 0.4149 | valid_acc: 0.9140 |
| Epoch | 41 | train_loss: 0.0562 | train_acc: 1.0000 | valid_loss: 0.4129 | valid_acc: 0.9180 |
| Epoch | 42 | train_loss: 0.0410 | train_acc: 1.0000 | valid_loss: 0.4836 | valid_acc: 0.9180 |
| Epoch | 43 | train_loss: 0.0467 | train_acc: 0.9792 | valid_loss: 0.5126 | valid_acc: 0.9120 |
| Epoch | 44 | train_loss: 0.0340 | train_acc: 1.0000 | valid_loss: 0.5021 | valid_acc: 0.9140 |
| Epoch | 45 | train_loss: 0.0400 | train_acc: 0.9792 | valid_loss: 0.5190 | valid_acc: 0.9140 |
| Epoch | 46 | train_loss: 0.0438 | train_acc: 1.0000 | valid_loss: 0.4701 | valid_acc: 0.9120 |

→ Final test accuracy: 0.9140

Retraining best model for width=256 (lr=8.24e-04, wd=1.26e-03, drop=0.201)

|       |    |                    |                   |                    |                   |
|-------|----|--------------------|-------------------|--------------------|-------------------|
| Epoch | 1  | train_loss: 0.8733 | train_acc: 0.7917 | valid_loss: 0.5439 | valid_acc: 0.7520 |
| Epoch | 2  | train_loss: 0.6117 | train_acc: 0.8333 | valid_loss: 0.5724 | valid_acc: 0.7760 |
| Epoch | 3  | train_loss: 0.4826 | train_acc: 0.6875 | valid_loss: 0.4692 | valid_acc: 0.7880 |
| Epoch | 4  | train_loss: 0.4302 | train_acc: 0.8125 | valid_loss: 0.4494 | valid_acc: 0.8260 |
| Epoch | 5  | train_loss: 0.3887 | train_acc: 0.8750 | valid_loss: 0.4621 | valid_acc: 0.8560 |
| Epoch | 6  | train_loss: 0.2954 | train_acc: 0.9167 | valid_loss: 0.4744 | valid_acc: 0.8600 |
| Epoch | 7  | train_loss: 0.2308 | train_acc: 0.9792 | valid_loss: 0.4321 | valid_acc: 0.8740 |
| Epoch | 8  | train_loss: 0.1249 | train_acc: 1.0000 | valid_loss: 0.4342 | valid_acc: 0.8780 |
| Epoch | 9  | train_loss: 0.1123 | train_acc: 1.0000 | valid_loss: 0.4175 | valid_acc: 0.8740 |
| Epoch | 10 | train_loss: 0.1208 | train_acc: 0.9792 | valid_loss: 0.4903 | valid_acc: 0.8680 |
| Epoch | 11 | train_loss: 0.0955 | train_acc: 0.9792 | valid_loss: 0.4577 | valid_acc: 0.8420 |
| Epoch | 12 | train_loss: 0.0707 | train_acc: 1.0000 | valid_loss: 0.4173 | valid_acc: 0.8900 |
| Epoch | 13 | train_loss: 0.0447 | train_acc: 1.0000 | valid_loss: 0.5469 | valid_acc: 0.8800 |
| Epoch | 14 | train_loss: 0.0238 | train_acc: 1.0000 | valid_loss: 0.4778 | valid_acc: 0.8940 |
| Epoch | 15 | train_loss: 0.0268 | train_acc: 1.0000 | valid_loss: 0.4093 | valid_acc: 0.8920 |
| Epoch | 16 | train_loss: 0.0254 | train_acc: 1.0000 | valid_loss: 0.4563 | valid_acc: 0.8940 |
| Epoch | 17 | train_loss: 0.0203 | train_acc: 1.0000 | valid_loss: 0.4703 | valid_acc: 0.9060 |
| Epoch | 18 | train_loss: 0.0217 | train_acc: 1.0000 | valid_loss: 0.4868 | valid_acc: 0.8900 |
| Epoch | 19 | train_loss: 0.0443 | train_acc: 0.9792 | valid_loss: 0.5227 | valid_acc: 0.8740 |
| Epoch | 20 | train_loss: 0.0312 | train_acc: 0.9792 | valid_loss: 0.4929 | valid_acc: 0.8820 |
| Epoch | 21 | train_loss: 0.0493 | train_acc: 1.0000 | valid_loss: 0.4827 | valid_acc: 0.9040 |
| Epoch | 22 | train_loss: 0.0334 | train_acc: 1.0000 | valid_loss: 0.4383 | valid_acc: 0.8940 |
| Epoch | 23 | train_loss: 0.0373 | train_acc: 1.0000 | valid_loss: 0.4280 | valid_acc: 0.9020 |
| Epoch | 24 | train_loss: 0.0266 | train_acc: 1.0000 | valid_loss: 0.4936 | valid_acc: 0.8980 |
| Epoch | 25 | train_loss: 0.0130 | train_acc: 0.9792 | valid_loss: 0.4322 | valid_acc: 0.8940 |
| Epoch | 26 | train_loss: 0.0238 | train_acc: 1.0000 | valid_loss: 0.4996 | valid_acc: 0.9060 |
| Epoch | 27 | train_loss: 0.0140 | train_acc: 1.0000 | valid_loss: 0.4760 | valid_acc: 0.8920 |
| Epoch | 28 | train_loss: 0.0070 | train_acc: 1.0000 | valid_loss: 0.5023 | valid_acc: 0.8960 |
| Epoch | 29 | train_loss: 0.0376 | train_acc: 1.0000 | valid_loss: 0.4259 | valid_acc: 0.8880 |

→ Final test accuracy: 0.8960

Retraining best model for width=1028 (lr=3.63e-04, wd=6.72e-04, drop=0.310)

|       |    |                    |                   |                    |                   |
|-------|----|--------------------|-------------------|--------------------|-------------------|
| Epoch | 1  | train_loss: 0.8422 | train_acc: 0.7292 | valid_loss: 0.5590 | valid_acc: 0.7600 |
| Epoch | 2  | train_loss: 0.6023 | train_acc: 0.8333 | valid_loss: 0.5106 | valid_acc: 0.7780 |
| Epoch | 3  | train_loss: 0.4925 | train_acc: 0.7917 | valid_loss: 0.4832 | valid_acc: 0.7980 |
| Epoch | 4  | train_loss: 0.4398 | train_acc: 0.7917 | valid_loss: 0.6242 | valid_acc: 0.7660 |
| Epoch | 5  | train_loss: 0.4449 | train_acc: 0.8333 | valid_loss: 0.4786 | valid_acc: 0.8320 |
| Epoch | 6  | train_loss: 0.3431 | train_acc: 0.8542 | valid_loss: 0.4934 | valid_acc: 0.7420 |
| Epoch | 7  | train_loss: 0.2948 | train_acc: 0.8958 | valid_loss: 0.4395 | valid_acc: 0.8520 |
| Epoch | 8  | train_loss: 0.4378 | train_acc: 0.8750 | valid_loss: 0.4901 | valid_acc: 0.8240 |
| Epoch | 9  | train_loss: 0.2684 | train_acc: 0.9375 | valid_loss: 0.4559 | valid_acc: 0.8660 |
| Epoch | 10 | train_loss: 0.2340 | train_acc: 0.9792 | valid_loss: 0.3819 | valid_acc: 0.8880 |
| Epoch | 11 | train_loss: 0.1533 | train_acc: 0.9375 | valid_loss: 0.3809 | valid_acc: 0.8900 |
| Epoch | 12 | train_loss: 0.1065 | train_acc: 0.9792 | valid_loss: 0.3934 | valid_acc: 0.8940 |
| Epoch | 13 | train_loss: 0.1048 | train_acc: 0.9792 | valid_loss: 0.5324 | valid_acc: 0.8380 |
| Epoch | 14 | train_loss: 0.1187 | train_acc: 1.0000 | valid_loss: 0.3843 | valid_acc: 0.8940 |
| Epoch | 15 | train_loss: 0.1401 | train_acc: 1.0000 | valid_loss: 0.3951 | valid_acc: 0.9020 |
| Epoch | 16 | train_loss: 0.0636 | train_acc: 0.9792 | valid_loss: 0.3666 | valid_acc: 0.9100 |
| Epoch | 17 | train_loss: 0.0633 | train_acc: 0.9792 | valid_loss: 0.5497 | valid_acc: 0.8880 |
| Epoch | 18 | train_loss: 0.0565 | train_acc: 0.9792 | valid_loss: 0.5360 | valid_acc: 0.8980 |
| Epoch | 19 | train_loss: 0.0534 | train_acc: 0.9583 | valid_loss: 0.3904 | valid_acc: 0.8920 |
| Epoch | 20 | train_loss: 0.0600 | train_acc: 0.9583 | valid_loss: 0.4391 | valid_acc: 0.8780 |

|          |                    |                   |                    |                   |
|----------|--------------------|-------------------|--------------------|-------------------|
| Epoch 21 | train_loss: 0.0964 | train_acc: 0.9792 | valid_loss: 0.3970 | valid_acc: 0.8900 |
| Epoch 22 | train_loss: 0.0472 | train_acc: 0.9792 | valid_loss: 0.3676 | valid_acc: 0.8980 |
| Epoch 23 | train_loss: 0.0632 | train_acc: 1.0000 | valid_loss: 0.4152 | valid_acc: 0.9020 |
| Epoch 24 | train_loss: 0.0339 | train_acc: 1.0000 | valid_loss: 0.3790 | valid_acc: 0.9060 |
| Epoch 25 | train_loss: 0.0296 | train_acc: 1.0000 | valid_loss: 0.4438 | valid_acc: 0.9180 |
| Epoch 26 | train_loss: 0.0254 | train_acc: 0.9792 | valid_loss: 0.4455 | valid_acc: 0.8960 |
| Epoch 27 | train_loss: 0.0188 | train_acc: 1.0000 | valid_loss: 0.4525 | valid_acc: 0.9060 |
| Epoch 28 | train_loss: 0.0220 | train_acc: 0.9792 | valid_loss: 0.4177 | valid_acc: 0.9100 |
| Epoch 29 | train_loss: 0.0395 | train_acc: 1.0000 | valid_loss: 0.4350 | valid_acc: 0.8840 |
| Epoch 30 | train_loss: 0.0403 | train_acc: 1.0000 | valid_loss: 0.5493 | valid_acc: 0.8720 |
| Epoch 31 | train_loss: 0.0509 | train_acc: 0.9792 | valid_loss: 0.5044 | valid_acc: 0.9140 |
| Epoch 32 | train_loss: 0.0391 | train_acc: 1.0000 | valid_loss: 0.3858 | valid_acc: 0.9040 |
| Epoch 33 | train_loss: 0.0484 | train_acc: 0.9792 | valid_loss: 0.3826 | valid_acc: 0.9100 |
| Epoch 34 | train_loss: 0.0304 | train_acc: 0.9792 | valid_loss: 0.3889 | valid_acc: 0.9200 |
| Epoch 35 | train_loss: 0.0262 | train_acc: 0.9792 | valid_loss: 0.3810 | valid_acc: 0.9080 |
| Epoch 36 | train_loss: 0.0473 | train_acc: 0.9792 | valid_loss: 0.3535 | valid_acc: 0.9100 |
| Epoch 37 | train_loss: 0.0255 | train_acc: 1.0000 | valid_loss: 0.3926 | valid_acc: 0.9040 |
| Epoch 38 | train_loss: 0.0206 | train_acc: 1.0000 | valid_loss: 0.4444 | valid_acc: 0.9040 |
| Epoch 39 | train_loss: 0.0281 | train_acc: 1.0000 | valid_loss: 0.4035 | valid_acc: 0.9140 |
| Epoch 40 | train_loss: 0.0224 | train_acc: 0.9792 | valid_loss: 0.4347 | valid_acc: 0.9100 |
| Epoch 41 | train_loss: 0.0161 | train_acc: 1.0000 | valid_loss: 0.3672 | valid_acc: 0.9180 |
| Epoch 42 | train_loss: 0.0181 | train_acc: 1.0000 | valid_loss: 0.4383 | valid_acc: 0.9160 |
| Epoch 43 | train_loss: 0.0059 | train_acc: 1.0000 | valid_loss: 0.5395 | valid_acc: 0.8940 |
| Epoch 44 | train_loss: 0.0086 | train_acc: 1.0000 | valid_loss: 0.4619 | valid_acc: 0.9140 |
| Epoch 45 | train_loss: 0.0167 | train_acc: 1.0000 | valid_loss: 0.3586 | valid_acc: 0.9300 |
| Epoch 46 | train_loss: 0.0120 | train_acc: 1.0000 | valid_loss: 0.4387 | valid_acc: 0.9220 |
| Epoch 47 | train_loss: 0.0073 | train_acc: 1.0000 | valid_loss: 0.4449 | valid_acc: 0.9120 |
| Epoch 48 | train_loss: 0.0080 | train_acc: 1.0000 | valid_loss: 0.4965 | valid_acc: 0.9080 |
| Epoch 49 | train_loss: 0.0105 | train_acc: 0.9583 | valid_loss: 0.4143 | valid_acc: 0.9180 |
| Epoch 50 | train_loss: 0.0233 | train_acc: 0.9792 | valid_loss: 0.3714 | valid_acc: 0.9260 |
| Epoch 51 | train_loss: 0.0426 | train_acc: 1.0000 | valid_loss: 0.4058 | valid_acc: 0.9220 |
| Epoch 52 | train_loss: 0.0233 | train_acc: 1.0000 | valid_loss: 0.4089 | valid_acc: 0.9100 |
| Epoch 53 | train_loss: 0.0112 | train_acc: 0.9792 | valid_loss: 0.3874 | valid_acc: 0.9140 |
| Epoch 54 | train_loss: 0.0301 | train_acc: 1.0000 | valid_loss: 0.5058 | valid_acc: 0.9180 |
| Epoch 55 | train_loss: 0.0127 | train_acc: 1.0000 | valid_loss: 0.4928 | valid_acc: 0.9060 |
| Epoch 56 | train_loss: 0.0247 | train_acc: 1.0000 | valid_loss: 0.3506 | valid_acc: 0.8980 |
| Epoch 57 | train_loss: 0.0345 | train_acc: 1.0000 | valid_loss: 0.3231 | valid_acc: 0.9160 |
| Epoch 58 | train_loss: 0.0191 | train_acc: 1.0000 | valid_loss: 0.3719 | valid_acc: 0.9240 |
| Epoch 59 | train_loss: 0.0128 | train_acc: 1.0000 | valid_loss: 0.3830 | valid_acc: 0.9060 |

→ Final test accuracy: 0.9260

```
=====
Final test accuracy by width:
hidden_size  best_val_acc  test_accuracy
         4         0.904         0.902
         8         0.908         0.784
        32         0.906         0.914
       256         0.914         0.896
      1028         0.916         0.926
```

## Visualizing Results

```
In [10]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Panel 1: test accuracy vs width
axes[0].plot(final_df["hidden_size"], final_df["test_accuracy"], marker='o', color='steelblue')
axes[0].set_xscale("log")
axes[0].set_xlabel("Hidden Size (log scale)")
axes[0].set_ylabel("Test Accuracy")
axes[0].set_title("Test Accuracy vs LSTM Width")
axes[0].grid(True, alpha=0.3)

# Panel 2: best learning rate vs width
axes[1].plot(final_df["hidden_size"], final_df["learning_rate"], marker='s', color='darkorange')
axes[1].set_xscale("log")
axes[1].set_yscale("log")
axes[1].set_xlabel("Hidden Size (log scale)")
axes[1].set_ylabel("Best Learning Rate (log scale)")
axes[1].set_title("Optimal Learning Rate vs LSTM Width")
axes[1].grid(True, alpha=0.3)

# Panel 3: best dropout vs width
axes[2].plot(final_df["hidden_size"], final_df["dropout"], marker='^', color='seagreen')
axes[2].set_xscale("log")
axes[2].set_xlabel("Hidden Size (log scale)")
axes[2].set_ylabel("Best Dropout")
```

```

axes[2].set_title("Optimal Dropout vs LSTM Width")
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("width_sweep_summary.png", dpi=150, bbox_inches='tight')
plt.show()
print("Figure saved to width_sweep_summary.png")

```

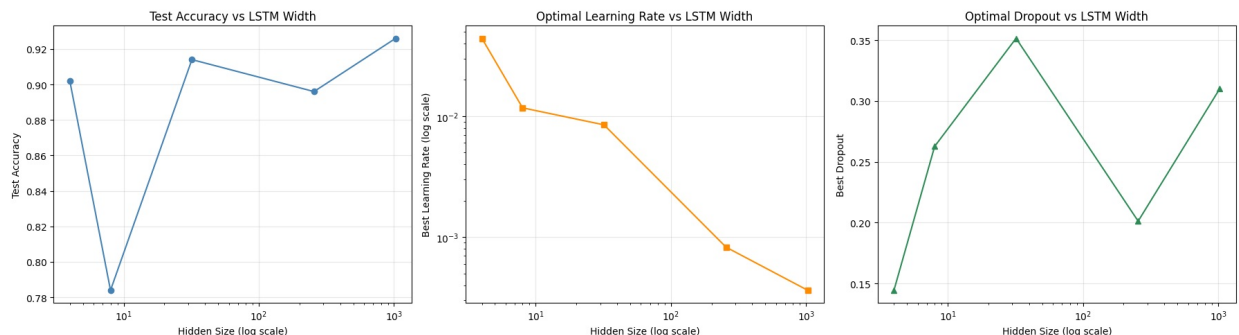


Figure saved to width\_sweep\_summary.png

```

In [11]: # Optuna vis for the largest study (1028) and the mid-range study (32)
for w in [32, 1028]:
    study = all_studies[w]
    print(f"\n--- Optuna plots for hidden_size = {w} ---")

    plot_param_importances(study).show()

    fig = plot_slice(study)
    fig.update_traces(marker=dict(colors=pc.qualitative.Set1),
                      selector=dict(type="scatter"))
    fig.show()

    fig = plot_parallel_coordinate(study)
    fig.update_traces(line=dict(colors=pc.qualitative.Set1), selector=dict(type="parcoords"))
    fig.show()

```

--- Optuna plots for hidden\_size = 32 ---

[Interactive Plotly charts rendered in Colab: hyperparameter importances, slice plot, parallel coordinate plot]

--- Optuna plots for hidden\_size = 1028 ---

[Interactive Plotly charts rendered in Colab: hyperparameter importances, slice plot, parallel coordinate plot]

## Discussion: What Do the Results Tell Us?

### Overall Accuracy Pattern

The results reveal a striking pattern: accuracy does *not* plateau early as one might expect — instead it trends upward across the entire width range, from 0.902 at width 4 to 0.926 at width 1028, with the notable exception of width 8 (0.784), which appears to be an outlier likely due to the stochastic nature of Optuna with only 15 trials finding a suboptimal configuration for that specific width. The absence of a clear plateau suggests that even at width 1028, the LSTM is still extracting marginal value from additional capacity. This is somewhat surprising — with only ~1,264 training examples, one might expect large networks to quickly overfit and underperform.

### Optimal Hyperparameters vs Width

The learning rate shows a very clear and consistent monotonic decrease as width increases — from 0.044 at width 4 down to 0.00036 at width 1028, a decrease of roughly two orders of magnitude. This is exactly the pattern predicted by optimization theory: wider networks have more parameters and sharper loss landscapes, requiring smaller steps to avoid divergence. This pattern held without exception across all five widths.

Weight decay showed no consistent trend, hovering in the range 1e-5 to 1e-3 across all widths. This suggests that L2 regularization is less critical than dropout for controlling overfitting in this architecture, given the relatively small dataset.

Dropout increased from 0.144 at width 4 to a peak of 0.351 at width 32, then dropped somewhat for widths



256 and 1028 (0.201 and 0.310 respectively). The general upward pressure on dropout with width is consistent with the expectation that wider networks overfit more — but the non-monotone behavior at 256 and 1028 may reflect an interaction with the very small learning rates chosen there, which already act as implicit regularization by taking small steps.

Patience did not show a consistent trend with width, ranging from 9 to 23 epochs. This is somewhat surprising — wider models were not noticeably slower to converge.

### BERT Embeddings vs LSTM Architecture

The most striking finding is that a width-4 LSTM — with just 8 total hidden units (4 per direction) — achieves 90.2% test accuracy. This is only 2.4 percentage points below the best model (width 1028, 92.6%). A width-4 bidirectional LSTM has almost no capacity to learn complex temporal patterns on its own; essentially all of the representational work must be done by the frozen BERT embeddings. This is strong evidence that the BERT embedding layer, not the LSTM architecture, is the dominant contributor to accuracy on this task. The LSTM's role appears to be primarily to aggregate the sequence of token embeddings into a single fixed-length vector for classification — a relatively simple operation that even a tiny LSTM can perform adequately. This finding has a practical implication: for tasks with limited data like this one, investing in better embeddings (e.g., fine-tuning BERT rather than freezing it) is likely to yield larger returns than increasing LSTM width.

```
In [13]: WIDTHS      = [4, 8, 32, 256, 1028]
         NUM_LAYERS = 2

         width_results = [
             {"hidden_size": 4,      "best_val_acc": 0.904, "learning_rate": 0.044020, "weight_decay": 0.000586,
             {"hidden_size": 8,      "best_val_acc": 0.908, "learning_rate": 0.011712, "weight_decay": 0.000015,
             {"hidden_size": 32,     "best_val_acc": 0.906, "learning_rate": 0.008472, "weight_decay": 0.000011,
             {"hidden_size": 256,    "best_val_acc": 0.914, "learning_rate": 0.000824, "weight_decay": 0.001256,
             {"hidden_size": 1028,   "best_val_acc": 0.916, "learning_rate": 0.000363, "weight_decay": 0.000672,
         ]
         print("width_results restored from prior Optuna run.")
```

## Extra Credit: Are Some Sentences Intrinsically Difficult to Classify?

For each sample in the combined validation+test set, we calculate how many of the retrained models (one per width) classified it incorrectly. Texts that are consistently misclassified across all models are likely intrinsically difficult.

```
In [14]: combined_texts = val_texts + final_test_texts
         combined_labels = val_labels + final_test_labels
         max_seq = max(val_embeds.shape[1], test_embeds.shape[1])

         def pad_to(tensor, target_len):
             pad_size = target_len - tensor.shape[1]
             if pad_size == 0:
                 return tensor
             pad = torch.zeros(tensor.shape[0], pad_size, tensor.shape[2])
             return torch.cat([tensor, pad], dim=1)

         combined_embeds = torch.cat([
             pad_to(val_embeds, max_seq),
             pad_to(test_embeds, max_seq)
         ], dim=0)
         n_samples = len(combined_labels)
         per_sample_correct = np.zeros((n_samples, len(WIDTHS)), dtype=int)

         for col_idx, row in enumerate(width_results):
             w = row["hidden_size"]
             drop = row["dropout"]

             model = LSTMPreEmbedded(hidden_size=w, num_layers=NUM_LAYERS, dropout=drop)
             optimizer = Adam(model.parameters(), lr=row["learning_rate"], weight_decay=row["weight_decay"])
             module = SimpleModule.classification(model, optimizer=optimizer, num_classes=3)

             ckpt_dir = f'ckpt/ec_w{w}'
             shutil.rmtree(ckpt_dir, ignore_errors=True)
             checkpoint_cb = ModelCheckpoint(
```

```

        monitor='valid_loss', mode='min', save_top_k=1, dirpath=ckpt_dir)
early_stop_cb = EarlyStopping(
    monitor='valid_accuracy', mode='max', patience=row["patience"])

trainer = Trainer(
    deterministic=True, max_epochs=200,
    logger=False, enable_progress_bar=False, enable_model_summary=False,
    callbacks=[ErrorTracker(), checkpoint_cb, early_stop_cb]
)
trainer.fit(module, datamodule=seq_dm_emb)

# best checkpoint
best_ckpt = checkpoint_cb.best_model_path
model.eval()

all_preds = []
loader = DataLoader(TensorDataset(combined_embeds), batch_size=64)
with torch.no_grad():
    for (batch_x,) in loader:
        logits = model(batch_x)
        preds = logits.argmax(dim=1).cpu().numpy()
        all_preds.extend(preds)

all_preds = np.array(all_preds)
correct = (all_preds == np.array(combined_labels)).astype(int)
per_sample_correct[:, col_idx] = correct

shutil.rmtree(ckpt_dir, ignore_errors=True)
del model, module, trainer
gc.collect()
torch.cuda.empty_cache()
print(f"Width {w:4d} - overall accuracy on combined set: {correct.mean():.4f}")

```

```

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero: Tip: For seamless cloud logging and experiment tracking,
try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metric
s and artifacts automatically to the Lightning Experiments platform.

```

```

INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
/usr/local/lib/python3.12/dist-packages/pytorch_lightning/utilities/pytree.py:21: `isinstance(treespec
c, LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec)` and `treespec.is_leaf()` instead.

```

```

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero: Tip: For seamless cloud logging and experiment tracking,
try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metric
s and artifacts automatically to the Lightning Experiments platform.
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

```

```
Width 4 - overall accuracy on combined set: 0.8810
```

```

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero: Tip: For seamless cloud logging and experiment tracking,
try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metric
s and artifacts automatically to the Lightning Experiments platform.
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

```

```
Width 8 - overall accuracy on combined set: 0.8740
```

```

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero: Tip: For seamless cloud logging and experiment tracking,
try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metric
s and artifacts automatically to the Lightning Experiments platform.
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

```

```
Width 32 - overall accuracy on combined set: 0.9140
```

```
Width 256 - overall accuracy on combined set: 0.9130
```

```

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero: Tip: For seamless cloud logging and experiment tracking,
try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metric
s and artifacts automatically to the Lightning Experiments platform.
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

```

```
Width 1028 - overall accuracy on combined set: 0.8990
```

```

In [15]: # accuracy per sample
sample_accuracy = per_sample_correct.mean(axis=1)

hard_indices = np.argsort(sample_accuracy)

```

```

print("Distribution of per-sample accuracy across models:")
print(pd.Series(sample_accuracy).describe())
print()

always_wrong = (sample_accuracy == 0).sum()
mostly_wrong = (sample_accuracy <= 0.2).sum()
print(f"Samples correctly classified by 0 models : {always_wrong}")
print(f"Samples correctly classified by ≤20% models: {mostly_wrong}")

```

Distribution of per-sample accuracy across models:

```

count    1000.000000
mean      0.896200
std       0.235714
min       0.000000
25%      1.000000
50%      1.000000
75%      1.000000
max       1.000000
dtype: float64

```

Samples correctly classified by 0 models : 33

Samples correctly classified by ≤20% models: 51

```

In [16]: # print the 5 most difficult examples
print("="*70)
print("TOP 5 MOST DIFFICULT SAMPLES")
print("="*70)
label_names = {0: "negative", 1: "neutral", 2: "positive"}

for rank, idx in enumerate(hard_indices[:5]):
    text = combined_texts[idx]
    label = combined_labels[idx]
    acc = sample_accuracy[idx]
    print(f"\nRank {rank+1} - accuracy across models: {acc:.2f}")
    print(f"True label : {label_names[label]}")
    print(f"Text      : {text}")
    print("-"*70)

```

=====

TOP 5 MOST DIFFICULT SAMPLES

=====

Rank 1 - accuracy across models: 0.00

True label : negative

Text : ADP News - Apr 22 , 2009 - Finnish business information systems developer Solteq Oyj HEL : STQ1V said today its net loss widened to EUR 189,000 USD 245,000 for the first quarter of 2009 from EUR 10,000 for the same peri

-----

Rank 2 - accuracy across models: 0.00

True label : positive

Text : Last year SysOpen Digia invested in IBM product know-how in the building of portal and trading place systems and successfully implemented customer solutions supported by it .

-----

Rank 3 - accuracy across models: 0.00

True label : positive

Text : With CapMan as a partner , we will be able to further develop our business and continue to focus on providing quality restaurant services for our customers , ' ' says Christopher Wynne , CEO of Papa John 's Russia .

-----

Rank 4 - accuracy across models: 0.00

True label : negative

Text : `` Those uncertainties cloud the long-term outlook . ''

-----

Rank 5 - accuracy across models: 0.00

True label : negative

Text : In Finland , the city of Forssa has said it will not pay compensation to food industry companies HK Ruokatalo and Atria for the lye leak into tap water that occurred in March 2008 .

-----

## Discussion: Intrinsically Difficult Texts

### Evidence for Intrinsic Difficulty

Yes — there is clear evidence that some texts are intrinsically more difficult to classify. The per-sample accuracy distribution is strongly bimodal: the median and 75th percentile are both 1.0 (most samples are correctly classified by all 5 models), yet 33 samples were misclassified by every single model, and 51 samples were misclassified by at least 4 out of 5 models. A standard deviation of 0.236 on a 0–1 scale confirms the distribution is far from uniform. If difficulty were random (just noise), we would expect a smooth distribution rather than this sharp split between easy and hard samples.

### The 5 Most Difficult Examples

All five hardest samples had 0.00 accuracy — misclassified by all five models. Looking at them:

**Rank 1 (true: negative)** — A report of a net loss widening from EUR 10,000 to EUR 189,000. This is genuinely negative, but the raw numbers are small (suggesting a small company), and the phrasing "net loss widened" is factual rather than emotionally charged. The BERT token-level embeddings may struggle with the implicit comparison required to understand "widening" as negative.

**Rank 2 (true: positive)** — A statement about a company investing in IBM product know-how and successfully implementing customer solutions. This reads as mildly positive but is written in a very neutral, matter-of-fact corporate tone. The word "successfully" is the only overt signal, and it appears mid-sentence without emphasis.

**Rank 3 (true: positive)** — A CEO quote about partnering with CapMan and focusing on quality restaurant services. This is typical boilerplate corporate language — the positive sentiment is implicit (partnerships are good for business) rather than explicit. The model likely treats it as neutral.

**Rank 4 (true: negative)** — A single short sentence: *"Those uncertainties cloud the long-term outlook."* This is an extremely short, context-free snippet. Without knowing what "those uncertainties" refer to, the statement is ambiguous — and the frozen BERT embedding has no document-level context to draw on. The metaphorical phrasing ("cloud the outlook") may also confuse models trained on more literal financial language.

**Rank 5 (true: negative)** — A statement that a city will not pay compensation to food companies for a lye contamination incident. This is negative for the companies, but requires understanding the implicit business impact of not receiving compensation — a level of inference that goes well beyond surface-level sentiment.

### Patterns and Hypotheses

The hard samples cluster around three failure modes: (1) **implicit vs. explicit sentiment** — positive or negative meaning that requires inference rather than keyword matching; (2) **context dependence** — short fragments that cannot be understood without the surrounding document; and (3) **corporate boilerplate** — language that is structurally neutral but semantically positive or negative to a domain expert. Notably, negative examples (only 13% of the training set) appear disproportionately among the hardest cases, consistent with the class imbalance making negative sentiment harder to learn.